

יחידה 9

סיבוכיות

ויעילות

מה סיבוכיות הזמן של קטעי הקוד הבאים ?

```
1. for (j=1; j<n; j++)  
    basic_step;
```

$O(n)$

```
2. for (i=n; i>1; i--)  
    for (j=0; j<n; j++)  
        basic_step;
```

$O(n^2)$

```
3. for (i=1; i<10; i++)  
    basic_step;
```

$O(1)$

```
4. for (i=1; i<n/2; i++)  
    basic_step_1;  
    for (i=0; i<n; i++)  
        for (j=0; j<i; j++)  
            basic_step_2;
```

$O(n^2)$

```
5. for (i=0; i<n/2; i++)  
    for (j=0; j<15; j++)  
        basic_step;
```

$O(n)$

```
6. for (i=0; i<n; i++) {  
    basic_step_1;  
    for (j=1; j<n; j++)  
        basic_step_2;  
}
```

$O(n^2)$

```
for( int i = n; i > 0; i /= 2 ) {  
    for( int j = 1; j < n; j *= 2 ) {  
        for( int k = 0; k < n; k += 2 ) {  
            ... // constant number of operations  
        }  
    }  
}
```

$O(n(\log n)^2)$

אֵיךְ מְמַיִנִים

```
public static void selectionSort(int[] array)
{
    for (int i = 0; i < array.length - 1; i++)
    {
        int index = i;
        for (int j = i + 1; j < array.length; j++)
            if (array[j] < array[index])
                index = j;

        int smallerNumber = array[index];
        array[index] = array[i];
        array[i] = smallerNumber;
    }
}
```

סיבוכיות $O(n^2)$

```
public static void bubbleSort(int array[]) {  
    int n = array.length;  
    int k;  
    for (int m = n; m >= 0; m--) {  
        for (int i = 0; i < n - 1; i++) {  
            k = i + 1;  
            if (array[i] > array[k]) {  
                swapNumbers(i, k, array);  
            }  
        }  
    }  
}
```

```
public static void swapNumbers(int i, int j, int[] array)  
{  
    int temp;  
    temp = array[i];  
    array[i] = array[j];  
    array[j] = temp;  
}
```

סיבוכיות $O(n^2)$

```
public static void insertionSort(int[] array)
{
    int temp;
    for (int i = 1; i < array.length; i++)
    {
        for(int j = i ; j > 0 ; j--)
        {
            if(array[j] < array[j-1])
                swapNumbers(j, j-1, array);
        }
    }
}
```

סיבוכיות $O(n^2)$

למה כדאי

למייין

האם איבר מסויים נמצא במערך מספרים ?

```
static int binarySearch(int[] array, int value)
{
    int left = 0, right = array.length - 1;
    while (left <= right)
    {
        System.out.println("Looking for " + value +
            " in range " + left + " .. " + right);
        int middle = (left + right) / 2;
        if (array[middle] == value)
            return middle;
        else if (array[middle] > value)
            right = middle - 1;
        else // (array[middle] < value)
            left = middle + 1;
    }
    return -1;
}
```

כשהמערך לא ממוין : צריך לעבור על כל המערך – סיבוכיות לינארית.
כשהמערך ממוין : חיפוש בינארי (אריה במדבר) – סיבוכיות לוגריתמית.

כתוב שיטה הבודקת האם מספר מסויים
מופיע יותר מפעם אחת במערך.

```
public static int findDuplicate(int[] array)
{
    int comparisons = 0;
    int duplicate = -1;
    for (int i = 0; i < array.length; ++i )
        for (int j = 0; j < array.length; ++j )
        {
            comparisons += 2;
            if ( i != j && array[i] == array[j])
                duplicate = i;
        }
    System.out.println(comparisons + " comparisons.");
    return duplicate;
}
```

```
public static int findDuplicateBetter(int[] array)
{
    int comparisons = 0;
    int duplicate = -1;
    for ( int i = 0; i < array.length; ++i )
    {
        for ( int j = 0; j < array.length; ++j )
        {
            comparisons += 2;
            if ( i != j && array[i] == array[j] )
            {
                duplicate = i;
                break;
            }
        }
        if (duplicate >= 0)
            break;
    }
    System.out.println(comparisons + " comparisons.");
    return duplicate;
}
```

```
public static int findDuplicateBest(int[] array)
{
    int comparisons = 0;
    for ( int i = 0; i < array.length; ++i )
        for ( int j = i + 1; j < array.length; ++j )
        {
            comparisons++;
            if (array[i] == array[j] )
            {
                System.out.println(comparisons + " comparisons.");
                return i;
            }
        }
    System.out.println(comparisons + " comparisons.");
    return -1;
}
```

בכל שלושת המקרים הסיבוכיות אותו דבר – ריבועית.

כתוב שיטה הבודקת האם מספר מסויים
מופיע יותר מפעם אחת במערך ממויין.

```
public static int findDuplicateInSortedArray(int[] array)
{
    int comparisons = 0;
    for ( int i = 0; i < array.length - 1; ++i )
    {
        comparisons++;
        if (array[i] == array[i + 1] )
        {
            System.out.println(comparisons + " comparisons.");
            return i;
        }
    }
    System.out.println(comparisons + " comparisons.");
    return -1;
}
```

הסיבוכיות עכשיו – לינארית, אבל אם המערך לא היה ממויין
– צריך היה למיינו תחילה בסיבוכיות של $O(n \log n)$.

```

public static void findMostCommonBetter(int []array)
{
    int maxCount = 0;
    int maxCountLocation = 0;

    for (int i = 0; i < array.length; i++)
    {
        if (array[i] == Integer.MAX_VALUE)
            continue;

        int count = 1;
        for (int j = i + 1; j < array.length; j++)
            if (array[i] == array[j])
            {
                count++;
                array[j] = Integer.MAX_VALUE;
            }

        if (count > maxCount)
        {
            maxCount = count;
            maxCountLocation = i;
        }
    }

    System.out.println("Most common : " +
        array[maxCountLocation] + " found " +
        maxCount + " times.");
}

```

כתוב שיטה הבודקת מהו המספר השכיח ביותר במערך.

```

public static void findMostCommon(int []array)
{
    int maxCount = 0;
    int maxCountLocation = 0;

    for (int i = 0; i < array.length; i++)
    {
        int count = 0;
        for (int j = 0; j < array.length; j++)
            if (array[i] == array[j])
                count++;

        if (count > maxCount)
        {
            maxCount = count;
            maxCountLocation = i;
        }
    }

    System.out.println("Most common : " +
        array[maxCountLocation] + " found " +
        maxCount + " times.");
}

```

בשני המקרים הסיבוכיות אותו דבר – ריבועית.

כתוב שיטה הבודקת מהו המספר השכיח ביותר במערך ממויין.

```
public static void findMostCommonInSortedArray(int []array)
{
    int comparisons = 0;
    int maxValue = array[0];
    int maxCount = 1;
    int tempValue = array[0];
    int tempCount = 1;
    for ( int i = 1; i < array.length; ++i )
    {
        comparisons++;
        if (array[i] == tempValue)
            tempCount++;
        else
        {
            if (tempCount > maxCount)
            {
                maxCount = tempCount;
                maxValue = tempValue;
            }
            tempValue = array[i];
            tempCount = 1;
        }
    }

    if (tempCount > maxCount)
    {
        maxCount = tempCount;
        maxValue = tempValue;
    }

    System.out.println(comparisons + " comparisons.");
    System.out.println("Most common : " + maxValue +
        " found " + maxCount + " times.");
}
```

הסיבוכיות עכשיו – לינארית, אבל אם המערך לא היה ממויין – צריך היה למיינו תחילה בסיבוכיות של $O(n \log n)$.

כתוב שיטה הבודקת האם יש שני איברים
במערך ממויין שסכומם שווה לערך נתון.

```
public static void twoSumSearchInSortedArray(int[] array, int value)
{
    int bottom = 0, top = array.length - 1;

    while (bottom < top)
    {
        System.out.println("Checking : " + array[bottom] +
            " at " + bottom + " plus " + array[top] +
            " at " + top + " sum to " + (array[bottom] + array[top]));
        if (array[bottom] + array[top] == value)
        {
            System.out.println("Found : " + array[bottom] +
                " at " + bottom + " plus " + array[top] +
                " at " + top + " sum to " + value);
            return;
        }
        if (array[bottom] + array[top] > value)
            top--;
        else // (array[bottom] + array[top] < value)
            bottom++;
    }
    System.out.println("Sum " + value + " was not found.");
}
```

סיבוכיות לינארית.

כתוב שיטה הבודקת האם יש שני איברים
במערך שסכומם שווה לערך נתון.

```
public static void twoSumSearch(int[] array, int value)
{
    int n = array.length;
    int count = 0;
    for (int i = 0; i < n; i++)
        for (int j = i+1; j < n; j++)
            if (array[i] + array[j] == value)
            {
                System.out.println("Found : " + array[i] +
                    " at " + i + " plus " + array[j] +
                    " at " + j + " sum to " + value);
                return;
            }

    System.out.println("Sum " + value + " was not found.");
}
```

סיבוכיות ריבועית.

נתונים 2 מערכים. כל מערך הינו מערך של מספרים שלמים שונים זה מזה
ונתונה השיטה הבאה:

```
public static boolean f(int[] arrayA, int[] arrayB) {  
    for (int i = 0; i < arrayA.length; i++) {  
        for (int j = 0; j < arrayB.length; j++) {  
            if (arrayA[ i ] >= arrayB[ j ])  
                return false;  
        }  
    }  
    return true;  
}
```

1. מה מבצעת שיטה זו ?
2. מהי סיבוכיות הזמן שלה (סדר-גודל) ?
3. ממשו את השיטה מחדש, באופן יעיל יותר (סיבוכיות הזמן צריכה להיות קטנה בסדר-גודל).
4. חשבו מהי סיבוכיות הזמן של השיטה שכתבתם (סדר-גודל) והסבירו את החישוב.
5. בהינתן שהמערכים ממוינים בסדר עולה, כיצד תממשו את השיטה באופן עוד יותר יעיל ? מהי סיבוכיות זמן הריצה של השיטה החדשה שכתבתם ?

נתונים 2 מערכים. כל מערך הינו מערך של מספרים שלמים שונים זה מזה
ונתונה השיטה הבאה:

```
public static boolean f(int[] arrayA, int[] arrayB) {  
    for (int i = 0; i < arrayA.length; i++) {  
        for (int j = 0; j < arrayB.length; j++) {  
            if (arrayA[i] >= arrayB[j])  
                return false;  
        }  
    }  
    return true;  
}
```

1. בודקת שכל האיברים במערך A קטנים מכל האיברים
במערך B

2. $O(n^2)$

```
public static boolean f2(int[] arrayA, int[] arrayB)  
{  
    int maxA = arrayA[0];  
    for (int i = 0; i < arrayA.length; i++)  
        if (arrayA[i] > maxA)  
            maxA = arrayA[i];  
  
    int minB = arrayB[0];  
    for (int i = 0; i < arrayB.length; i++)  
        if (arrayB[i] < minB)  
            minB = arrayB[i];  
  
    return (maxA < minB);  
}
```

4. $O(n)$ כי עוברים על כל מערך רק פעם אחת.

5. $O(1)$

```
public static boolean f3(int[] arrayA, int[] arrayB)  
{  
    return (arrayA[arrayA.length - 1] < arrayB[0]);  
}
```

1. מה מבצעת שיטה זו ?
2. מהי סיבוכיות הזמן שלה (סדר-גודל) ?
3. ממשו את השיטה מחדש, באופן יעיל יותר (סיבוכיות הזמן צריכה להיות קטנה בסדר-גודל).
4. חשבו מהי סיבוכיות הזמן של השיטה שכתבתם (סדר-גודל) והסבירו את החישוב.
5. בהינתן שהמערכים ממוינים בסדר עולה, כיצד תממשו את השיטה באופן עוד יותר יעיל ? מהי סיבוכיות זמן הריצה של השיטה החדשה שכתבתם ?

נתון מערך של מספרים שלמים שמהווה סדרה "עולה-יורדת", כלומר, עד איבר מסוים במערך, המספרים הולכים וגדלים, ומאיבר זה ואילך, הם הולכים וקטנים (ניתן להניח שאין שני איברים רצופים זהים).

לדוגמה: 1 15 200 211 123 8 6 5 3 1

ונתונה השיטה הבאה:

```
public static int f(int[] array)
{
    int tmp = array[0];
    for (int i = 1; i < array.length; i++) {
        if (array[i] > tmp){
            tmp = array[i];
        }
    }
    return tmp;
}
```

- 1. מה מבצעת שיטה זו?
- 2. מהי סיבוכיות הזמן שלה (סדר-גודל)?
- 3. ממשו את השיטה מחדש, באופן יעיל יותר (סיבוכיות הזמן צריכה להיות קטנה בסדר-גודל).
- 4. חשבו מהי סיבוכיות הזמן של השיטה שכתבתם (סדר-גודל) והסבירו את החישוב.
- 5. כתבו שיטה שמוצאת את האיבר המינימלי במערך המהווה סדרה "עולה-יורדת", וחשבו את סיבוכיות הזמן של שיטה זו.

1. מחזירה את האיבר הגדול במערך

2. $O(n)$

3.

```
public static int f2(int [] array)
{
    if (array[0] > array[1])
        return array[0];
    if (array[array.length - 1] > array[array.length - 2])
        return array[array.length - 1];

    int low=0, high=array.length - 1, mid = -1;
    while (low <= high)
    {
        mid = (low + high) / 2;
        if(array[mid+1] < array[mid] &&
            array[mid-1] < array[mid]) // found the maximum
            return array[mid];
        else if(array[mid+1] > array[mid]) // going down...
            low = mid + 1;
        else if(array[mid-1] > array[mid]) // going up...
            high = mid - 1;
        else
            System.out.println("ERROR !");
    }
    return array[mid];
}
```

4. $O(\log n)$ – חיפוש בינארי על המערך.

5. $O(1)$

```
public static int f3(int[] array)
{
    return Math.min(array[0], array[array.length - 1]);
}
```

נתון מערך של מספרים שלמים שמהווה סדרה "עולה-יורדת", כלומר, עד איבר מסוים במערך, המספרים הולכים וגדלים, ומאיבר זה ואילך, הם הולכים וקטנים (ניתן להניח שאין שני איברים רצופים זהים).

לדוגמה: 1 3 5 6 8 123 211 200 15 1

ונתונה השיטה הבאה:

```
public static int f(int[] array)
{
    int tmp = array[0];
    for (int i = 1; i < array.length; i++) {
        if (array[i] > tmp){
            tmp = array[i];
        }
    }
    return tmp;
}
```

1. מה מבצעת שיטה זו?
2. מהי סיבוכיות הזמן שלה (סדר-גודל)?
3. ממשו את השיטה מחדש, באופן יעיל יותר (סיבוכיות הזמן צריכה להיות קטנה בסדר-גודל).
4. חשבו מהי סיבוכיות הזמן של השיטה שכתבתם (סדר-גודל) והסבירו את החישוב.
5. כתבו שיטה שמוצאת את האיבר המינימלי במערך המהווה סדרה "עולה-יורדת", וחשבו את סיבוכיות הזמן של שיטה זו.

```
public boolean something(int[] a, int[] b)
{
    int i, j, N = a.length.
    for(j = 0 ; j < N ; j++)
        for(i = 0 ; i < N -1 ; i++)
            if(b[j] == a[i]+a[i+1])
                return true;
    return false;
}
```

א. מה עושה השיטה?

ב. מה סיבוכיות זמן הריצה של השיטה?

ג. בהנחה שמערך a ממויין בסדר עולה ממש (מערך b לא ממויין) -

כתוב את השיטה something מחדש כך שתעשה מה שעשתה השיטה מסעיף

א', אולם בזמן ריצה טוב יותר.

- א. השיטה מחפשת האם במערך B יש איבר ששווה לסכום שני איברים צמודים ב-A.
- ב. סיבוכיות ריבועית.

ג.

```
public static boolean something2(int[] a, int[] b)
{
    int middle;
    for (int j = 0 ; j < a.length ; j++)
    {
        int lower = 0;
        int upper = a.length - 1;
        while (lower <= upper)
        {
            middle = (lower + upper) / 2;
            if (a[middle] + a[middle + 1] == b[j])
                return true;
            if (middle + 1 == a.length)
                break;
            if (b[j] < (a[middle] + a[middle + 1]))
                upper = middle - 1;
            else
                lower = middle + 1;
        }
    } // for loop
    return false;
}
```

סיבוכיות ($O(n \log n)$)

```
public boolean something(int[] a, int[] b)
{
    int i, j, N = a.length;
    for(j = 0 ; j < N ; j++)
        for(i = 0 ; i < N - 1 ; i++)
            if(b[j] == a[i]+a[i+1])
                return true;
    return false;
}
```

א. מה עושה השיטה?

ב. מה סיבוכיות זמן הריצה של השיטה?

- ג. בהנחה שמערך a ממויין בסדר עולה ממש (מערך b לא ממויין) - כתוב את השיטה something מחדש כך שתעשה מה שעשתה השיטה מסעיף א', אולם בזמן ריצה טוב יותר.

```
public boolean what(int[] arr1, int[] arr2, int num)
{
    for(int i=0; i<arr1.length; i++)
        for(int j=0; j<arr2.length; j++)
            if(arr1[i] + arr2[j] == num)
                return true;
    return false;
}
```

א. בהנחה ששני המערכים arr1, arr2 מכילים ערכים שלמים
ממויינים בסדר עולה ממש, ו-num הוא מספר שלם, מה מבצעת
השיטה?

ב. מהו סדר גודל זמן ריצת השיטה?

ג. כתבו את השיטה what כך שתפתור את הבעיה בסיבוכיות זמן
ריצה קטנה מסיבוכיות הזמן של השיטה מסעיף א'. כתבו מהי
סיבוכיות זמן הריצה של השיטה שכתבתם.

```
public boolean what(int[] arr1, int[] arr2, int num)
{
    for(int i=0; i<arr1.length; i++)
        for(int j=0; j<arr2.length; j++)
            if(arr1[i] + arr2[j] == num)
                return true;
    return false;
}
```

א. בהנחה ששני המערכים arr1, arr2 מכילים ערכים שלמים
ממויינים בסדר עולה ממש, ו-num הוא מספר שלם, מה מבצעת
השיטה?

ב. מהו סדר גודל זמן ריצת השיטה?

ג. כתבו את השיטה what כך שתפתור את הבעיה בסיבוכיות זמן
ריצה קטנה מסיבוכיות הזמן של השיטה מסעיף א'. כתבו מהי
סיבוכיות זמן הריצה של השיטה שכתבתם.

```
public boolean what3(int[] arr1, int[] arr2, int num)
{
    int i = 0, j = arr2.length-1;
    while(i < arr1.length && j >= 0)
    {
        if(arr1[i] + arr2[j] > num)
            j--;
        else if(arr1[i] + arr2[j] < num)
            i++;
        else
            return true;
    }
    return false;
}
```

סיבוכיות לינארית!

א. השיטה בודקת האם קיים מספר במערך arr1 ומספר
במערך arr2 שסכומם הוא הערך של num.

ב. $O(n^2)$

ג.

```
public int binary2(int[] arr, int num)
{
    int top = arr.length - 1;
    int bottom = 0;
    int mid;

    while(top >= bottom)
    {
        mid = (top+bottom)/2;
        if(num > arr[mid])
            bottom = mid + 1;
        else if(num < arr[mid])
            top = mid - 1;
        else
            return mid;
    }

    return -1;
}
```

```
public boolean what2(int[] arr1, int[] arr2, int num)
{
    for(int i=0; i<arr1.length; i++)
        if(binary2(arr2, num-arr1[i]) != -1)
            return true;
    return false;
}
```

הסיבוכיות – $O(n \log n)$. האם אפשר יותר טוב?

הפעולה	זמן במערך ממוין	זמן במערך לא ממוין
הכנסת איבר למערך		
חישוב סכום האיבר הרביעי בגודלו עם האיבר השביעי בגודלו		
בדיקה אם איבר x נמצא במערך		
הוצאת המינימום מהמערך		
בדיקה אם יש זוג איברים במערך שסכומם שווה למספר כלשהו x .		
בדיקה מה ההפרש הגדול ביותר בין שני איברים כלשהם במערך		

שימו לב, אינכם צריכים לכתוב את האלגוריתם אליו אתם מתכוונים אלא רק את הסיבוכיות של האלגוריתם היעיל ביותר.

הפעולה	זמן במערך ממוין	זמן במערך לא ממוין
הכנסת איבר למערך	$O(n)$	$O(n)$
חישוב סכום האיבר הרביעי בגודלו עם האיבר השביעי בגודלו	$O(1)$	$O(n)$
בדיקה אם איבר x נמצא במערך	$O(\log n)$	$O(n)$
הוצאת המינימום מהמערך	$O(n)$	$O(n)$
בדיקה אם יש זוג איברים במערך שסכומם שווה למספר כלשהו x.	$O(n)$	$O(n^2)$
בדיקה מה ההפרש הגדול ביותר בין שני איברים כלשהם במערך	$O(1)$	$O(n)$

שימו לב, אינכם צריכים לכתוב את האלגוריתם אליו אתם מתכוונים אלא רק את הסיבוכיות של האלגוריתם היעיל ביותר.

לקט שאלות

כתבו שיטה שחתימתה –

```
public static int halfSum(int[] a)
```

השיטה מקבלת כפרמטר מערך של מספרים חיוביים (לא ממויין בהכרח) ומחזירה את האינדקס i במערך שסכום כל האיברים $a[0]+a[1]+\dots+a[i]$ שווה לסכום כל האיברים $a[i+1]+\dots+a[a.length-1]$.

אם לא קיים אינדקס כזה, השיטה תחזיר -1.

למשל – במערך $\{2, 4, 3, 1, 2, 3, 5\}$ השיטה תחזיר את האינדקס 3, כיוון שסכום המספרים $2+4+1+3$ שווה לסכום המספרים $2+3+5$.

```

public static int halfSumSquareComplexity(int[] a)
{
    for (int i = 0; i < a.length; i++)
    {
        int sum1 = 0, sum2 = 0;
        for (int j = 0; j <= i; j++)
            sum1 += a[j];
        for (int j = i + 1; j < a.length; j++)
            sum2 += a[j];
        if (sum1 == sum2)
            return i;
    }

    return -1;
}

```

כתבו שיטה שחתימתה –

```
public static int halfSum(int[] a)
```

השיטה מקבלת כפרמטר מערך של מספרים חיוביים (לא ממויין בהכרח) ומחזירה את האינדקס i במערך שסכום כל האיברים $a[0]+a[1]+\dots+a[i]$ שווה לסכום כל האיברים $a[i+1]+\dots+a[a.length-1]$.

אם לא קיים אינדקס כזה, השיטה תחזיר -1.

```

public static int halfSumLinearComplexity(int[] a)
{
    int totalSum = 0;
    for (int i = 0; i < a.length; i++)
        totalSum += a[i];
    int halfSum = 0;
    for (int i = 0; i < a.length; i++)
    {
        halfSum += a[i];
        if (halfSum * 2 == totalSum)
            return i;
        if (halfSum * 2 > totalSum)
            return -1;
    }

    return -1;
}

```

למשל – במערך $\{2, 4, 3, 1, 2, 3, 5\}$ השיטה תחזיר את האינדקס 3, כיוון שסכום המספרים $2+4+1+3$ שווה לסכום המספרים $2+3+5$.

נגדיר : זוג מספרים שלמים a ו-b נקראים **חברים** (friends) אם הם שונים זה מזה, וגם מתקיים התנאי שסכום המחלקים של a שווה ל-b וסכום המחלקים של b שווה ל-a. (המחלקים, לא כולל כמובן הערכים a ו-b עצמם.

לדוגמא:

המספרים 220 ו-284 הם חברים, כיוון ש :

סכום המחלקים של 220 הוא 284 :

$$1 + 2 + 4 + 5 + 10 + 11 + 20 + 22 + 44 + 55 + 110 = 284$$

וסכום המחלקים של 284 הוא 220 :

$$1 + 2 + 4 + 71 + 142 = 220$$

עליכם לכתוב שיטה סטטית בשם countFriends המקבלת מספר שלם חיובי n. השיטה צריכה להחזיר את מספר הזוגות של מספרים חברים ששניהם קטנים מ-n.
לדוגמא :

השיטה (countFriends(9000 תחזיר את הערך 5, שכן יש חמישה זוגות של מספרים חברים ששניהם קטנים מ-9000. והם :

220 , 284
1184 , 1210
2620 , 2924
5020 , 5564
6232 , 6368

חתימת השיטה היא :

```
public static int countFriends(int n)
```

השיטה שתכתבו צריכות להיות יעילה ככל הניתן.

שיטה שתעבוד בסיבוכיות גבוהה מזו הנדרשת (במקום או בזמן) לא תקבל את מירב הנקודות.

אפשר להשתמש בשיטות עזר שתכתבו, אבל זכרו שבחישוב סיבוכיות זמן הריצה צריך לקחת גם אותן בחשבון.

אסור להשתמש בשיטות ממחלקות אחרות (כמו Math)

אל תשכחו לתעד את מה שכתבתם!

```
private static int sumOfDivisors(int number)
```

```
{
```

```
// O(n) solution
```

```
/*
```

```
if (number <= 1)
```

```
    return 0;
```

```
int sum = 0;
```

```
for(int divisor = 1; divisor < number; divisor++)
```

```
    if (number % divisor == 0)
```

```
        sum += divisor;
```

```
return sum;
```

```
*/
```

```
// O( sqrt(n) ) = O( n ^ (1/2) ) solution
```

```
if (number <= 1)
```

```
    return 0;
```

```
int sum = 1;
```

```
for(int divisor = 2; (divisor * divisor) <= number; divisor++)
```

```
{
```

```
    int remainder = number % divisor;
```

```
    int quotient = number / divisor;
```

```
    if (remainder == 0) {
```

```
        if (quotient == divisor)
```

```
            // which means divisor * divisor == number
```

```
            sum += divisor;
```

```
        else
```

```
            sum += (divisor + quotient);
```

```
    }
```

```
}
```

```
return sum;
```

```
}
```

```
private static boolean isFriend(int a, int n)
```

```
{
```

```
    int b = sumOfDivisors(a);
```

```
    return (b > a && b < n && a == sumOfDivisors(b));
```

```
}
```

```
public static int countFriends(int n)
```

```
{
```

```
    int count = 0;
```

```
    for (int a = 1; a < n; a++)
```

```
        if (isFriend(a, n)) {
```

```
            count++;
```

```
            // Optional: print Friends Pair
```

```
            int b = sumOfDivisors(a);
```

```
            System.out.println(a + ", " + b);
```

```
        }
```

```
    return count;
```

```
}
```

סיבוכיות זמן $O(n^{1.5})$

נגדיר: זוג מספרים שלמים a ו-b נקראים חברים (friends) אם הם שונים זה מזה, וגם מתקיים

התנאי שסכום המחלקים של a שווה ל-b וסכום המחלקים של b שווה ל-a. (המחלקים, לא כולל

כמובן הערכים a ו-b עצמם).

לדוגמא:

המספרים 220 ו-284 הם חברים, כיוון ש:

סכום המחלקים של 220 הוא 284:

$1 + 2 + 4 + 5 + 10 + 11 + 20 + 22 + 44 + 55 + 110 = 284$

וסכום המחלקים של 284 הוא 220:

$1 + 2 + 4 + 71 + 142 = 220$

עליכם לכתוב שיטה סטטית בשם countFriends המקבלת מספר שלם חיובי n. השיטה צריכה

להחזיר את מספר הזוגות של מספרים חברים ששניהם קטנים מ-n.

לדוגמא:

השיטה countFriends(9000) תחזיר את הערך 5, שכן יש חמישה זוגות של מספרים חברים

ששניהם קטנים מ-9000. והם:

220, 284

1184, 1210

2620, 2924

5020, 5564

6232, 6368

חתימת השיטה היא:

```
public static int countFriends(int n)
```

השיטה שתכתבו צריכות להיות יעילה ככל הניתן.

שיטה שתעבוד בסיבוכיות גבוהה מזו הנדרשת (במקום או בזמן) לא תקבל את מירב הנקודות.

אפשר להשתמש בשיטות עזר שתכתבו, אבל זכרו שבחישוב סיבוכיות זמן הריצה צריך לקחת גם

אותן בחשבון.

אסור להשתמש בשיטות ממחלקות אחרות (כמו Math)

אל תשכחו לתעד את מה שכתבתם!

הגדרה: נאמר שמספר x הוא k -מוכל במספר y אם כל ספרה ב- x מופיעה גם ב- y למעט בדיוק k ספרות שמופיעות ב- x ולא מופיעות ב- y אם ספרות מופיעות יותר מפעם אחת ב- x , מספיק שאחת יופיע פעם אחת ב- y .

לדוגמא,

המספר 4342113 הוא 2-מוכל במספר 45111 כי חורף מהספרות 2 ו-1, כל הספרות במספר הראשון מופיעות גם במספר השני.

כנגד שיטה טכנית included המקבלת שני מספרים שלמים x ו- y , ומחזירה את הערך k כך ש- x הוא k -מוכל ב- y .

ניתן להניח ששני המספרים חיוביים

החימת השיטה היא:

```
public static int included (int x, int y)
```

שימו לב:

השיטה שתכתבו צריכה להיות יעילה ככל הניתן, גם מבחינת סיבוכיות הזמן וגם מבחינת סיבוכיות המקום. תשובה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבעיה תקבל מעט נקודות בלבד.

כתבו מה סיבוכיות הזמן וסיבוכיות המקום של השיטה שכתבתם.

אל תשכחו לתעד את מה שכתבתם!

חשוב מאוד - למרות שהמשתנים מטיפוס `int` מוגבלים בגודלם, ההתייחסות לאורך של x ושל y היא כאל n שאינו קבוע. כלומר, מעבר על הספרות של x או y הוא לינארי $O(n)$.

```

public static int included (int x,int y)
{
    boolean[] xDigits = {false,false,false,false,
        false,false,false,false,false,false};
    boolean[] yDigits = {false,false,false,false,
        false,false,false,false,false,false};
    while (x != 0)
    {
        xDigits[x%10] = true;
        x = x / 10;
    }
    while (y != 0)
    {
        yDigits[y%10] = true;
        y = y / 10;
    }
    int k = 0;
    for (int i = 0; i < 10; i++)
    {
        if (xDigits[i] && !yDigits[i])
        {
            System.out.println("Digit : " + i);
            k++;
        }
    }
    return k;
}

```

הגדרה: נאמר שמספר x הוא k-מוכל במספר y אם כל ספרה ב-x מופיעה גם ב-y למעט בדיוק k ספרות שמופיעות ב-x ולא מופיעות ב-y. אם ספרות מופיעות יותר מפעם אחת ב-x, מספיק לאחד יופיע פעם אחת ב-y.

לדוגמה, המספר 4342112 הוא 2-מוכל במספר 45111 כי חוץ מהספרות 2 ו-1, כל הספרות במספר הראשון מופיעות גם במספר השני.

כתבו שיטה טכנית included שמקבלת שני מספרים שלמים x ו-y, ומחזירה את הערך k כך ש-x הוא k-מוכל ב-y.

ניתן להניח ששני המספרים חיוביים.

חתימת השיטה היא:

```
public static int included (int x, int y)
```

שימו לב:

השיטה שתכתבו צריכה להיות יעילה ככל הניתן, גם מבחינת סיבוכיות הזמן וגם מבחינת סיבוכיות המקום. תשובה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבעיה תקבל מעט נקודות בלבד.

כתבו מה סיבוכיות הזמן וסיבוכיות המקום של השיטה שכתבתם.

אל תשכחו לתעד את מה שכתבתם!

חשוב מאוד - למרות שהמשתנים מטיפוס int מוגבלים בגודלם, ההתייחסות לאורך של x ושל y היא כאלו שאינם קבוע. כלומר, מעבר על הספרות של x או y הוא לינארי $O(n)$.

נתונה המחלקה IntVector הבאה המממשת מערך חד ממדי המכיל מספרים שלמים.

```
public class IntVector
{
    private int []_arr;

    /**
     * Constructor for objects of class IntVector
     */
    public IntVector(int size)
    {
        _arr= new int[size];
    }

    ...    //methods
}
```

נניח שבמערך, כל המספרים הם שלמים אי-שליליים.

הוסיפו למחלקה IntVector שיטה בשם sortByFour הממיינת את המערך באופן הבא:

- בתחילת המערך יופיעו כל המספרים שמתחלקים ב-4 ללא שארית.
- לאחריהם יופיעו כל המספרים שמתחלקים ב-4 עם שארית 1
- לאחריהם יופיעו כל המספרים שמתחלקים ב-4 עם שארית 2
- בסוף המערך יופיעו כל יתר המספרים (אלו שמתחלקים ב-4 עם שארית 3).

אין חשיבות לסדר הפנימי של המספרים בתוך כל קבוצה.

השיטה צריכה להיות יעילה ככל הניתן.

שיטה שתעבוד בסיבוכיות גבוהה מזו הנדרשת (במקום או בזמן) לא תקבל את מירב הנקודות.

נתונה המחלקה IntVector הבאה המממשת מערך חד ממדי המכיל מספרים שלמים.

```
public static void sortByFour(int[] array)
{
    int temp, x = 0;
    for(int i = 0; i < 4; i++)
    {
        for(int j = 0; j < array.length; j++)
        {
            if(array[j]%4 == i)
            {
                temp = array[x];
                array[x] = array[j];
                array[j] = temp;
                x++;
            }
        }
    }
}
```

```
public class IntVector
{
    private int []_arr;

    /**
     * Constructor for objects of class IntVector
     */
    public IntVector(int size)
    {
        _arr= new int[size];
    }

    ...    //methods
}
```

נניח שבמערך, כל המספרים הם שלמים אי-שליליים.

הוסיפו למחלקה IntVector שיטה בשם sortByFour הממיינת את המערך באופן הבא:

- בתחילת המערך יופיעו כל המספרים שמתחלקים ב-4 ללא שארית.
- לאחריהם יופיעו כל המספרים שמתחלקים ב-4 עם שארית 1
- לאחריהם יופיעו כל המספרים שמתחלקים ב-4 עם שארית 2
- בסוף המערך יופיעו כל יתר המספרים (אלו שמתחלקים ב-4 עם שארית 3).

אין חשיבות לסדר הפנימי של המספרים בתוך כל קבוצה.

השיטה צריכה להיות יעילה ככל הניתן.

שיטה שתעבוד בסיבוכיות גבוהה מזו הנדרשת (במקום או בזמן) לא תקבל את מירב הנקודות.

נגדיר:

נגיד שמערך arr הוא מערך k-מודולו ממוין אם הוא מערך של מספרים טבעיים (שלמים וחיוביים), שבו לכל שני אינדקסים i, j מתקיים שאם $i < j$ אזי $arr[i] \bmod k \leq arr[j] \bmod k$ (פעולת mod היא פעולת השארית שמבוצעת על-ידי האופרטור % ב-Java).

כתבו שיטה סטטית sortMod המקבלת כפרמטרים מערך של מספרים טבעיים a, ומספר טבעי k. השיטה צריכה למיין את המערך כך שיהיה k-מודולו ממוין.

לדוגמא, עבור המערך a להלן, ועבור $k = 10$

0	1	2	3	4	5	6	7	8
35	17	13	252	4	128	7	3	81

המערך ה-k-מודולו ממוין יכול להיות:

0	1	2	3	4	5	6	7	8
81	252	13	3	4	35	17	7	128

שימו לב שהמערך יכול להיות גם אחר. למשל הערכים בתא 2 ו-3 מוחלפים (או בתא 6 ו-7).

חתימת השיטה היא:

```
public static void sortMod(int a[], int k)
```

שימו לב:

השיטה שתכתבו צריכה להיות יעילה ככל הניתן, גם מבחינת סיבוכיות הזמן וגם מבחינת סיבוכיות המקום. תשובה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבעיה תקבל מעט נקודות בלבד.

k הוא קבוע. למרות זאת,

אסור להקצות מערך נוסף בשום גודל. גם לא בגודל k. גם לא להקצות רשימה

מקושרת בגודל k או k משתנים בודדים.

כתבו מה סיבוכיות הזמן וסיבוכיות המקום של השיטה שכתבתם.

אל תשכחו לתעד את מה שכתבתם!

```

public static void sortMod(int[] array, int k)
{
    int temp, x = 0;
    for(int i = 0; i < k; i++)
    {
        for(int j = 0; j < array.length; j++)
        {
            if(array[j] % k == i)
            {
                temp = array[x];
                array[x] = array[j];
                array[j] = temp;
                x++;
            }
        }
    }
}

```

נגדיר:
 נגיד שמערך arr הוא מערך k-מודולו ממוין אם הוא מערך של מספרים טבעיים (שלמים וחיוביים), שבו לכל שני אינדקסים i, j מתקיים שאם $i < j$ אזי $arr[i] \bmod k \leq arr[j] \bmod k$ (פעולת mod היא פעולת השארית שמבוצעת על-ידי האופרטור % ב-Java).

כתבו שיטה סטטית sortMod המקבלת כפרמטרים מערך של מספרים טבעיים a, ומספר טבעי k. השיטה צריכה למיין את המערך כך שיהיה k-מודולו ממוין.

לדוגמא, עבור המערך a להלן, ועבור $k = 10$

0	1	2	3	4	5	6	7	8
35	17	13	252	4	128	7	3	81

המערך ה-k-מודולו ממוין יכול להיות:

0	1	2	3	4	5	6	7	8
81	252	13	3	4	35	17	7	128

שימו לב שהמערך יכול להיות גם אחר. למשל הערכים בתא 2 ו-3 מוחלפים (או בתא 6 ו-7).

חתימת השיטה היא:

```
public static void sortMod(int a[], int k)
```

שימו לב:

השיטה שתכתבו צריכה להיות יעילה ככל הניתן, גם מבחינת סיבוכיות הזמן וגם מבחינת סיבוכיות המקום. תשובה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבעיה תקבל מעט נקודות בלבד.

k הוא קבוע. למרות זאת,

אסור להקצות מערך נוסף בשום גודל. גם לא בגודל k. גם לא להקצות רשימה מקושרת בגודל k או k משתנים בודדים.
 כתבו מה סיבוכיות הזמן וסיבוכיות המקום של השיטה שכתבתם.

אל תשכחו לתעד את מה שכתבתם!

```
public static boolean what (int n)
{
    return (f(n)%2 == 0);
}

public static int f (int n)
{
    int c = 1;
    for (int i=2; i<=n; i++)
    {
        if (n%i == 0)
            c++;
    }
    return c;
}
```

א. מה מבצעת השיטה what בהינתן לה מספר שלם חיובי n כלשהו. הסבירו בקצרה מה מבצעת השיטה ולא כיצד היא מבצעת זאת. כלומר, כתבו במפורש מה המשמעות של הערך המוחזר מהשיטה what. (4 נק').

ב. מהי סיבוכיות הזמן של השיטה what ? (2 נק')

ג. כתבו את השיטה what כך שתבצע את מה שביצעה בסעיף א בסיבוכיות זמן ריצה קטנה יותר. (17 נק')

שימו לב:

השיטה שתכתבו צריכה להיות יעילה ככל הניתן, גם מבחינת סיבוכיות הזמן וגם מבחינת סיבוכיות המקום. תשובה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבעיה תקבל מעט נקודות בלבד.

ד. מה סיבוכיות זמן הריצה של השיטה שכתבתם בסעיף ג? הסבירו תשובתכם. (2 נק')

אל תשכחו לתעד את מה שכתבתם!

האם מספר המחלקים של מספר נתון הוא זוגי או לא –
בדיקה האם יש לו שורש שהוא מספר שלם.

```
public static boolean myWhat(int n)
{
    int temp = (int)Math.sqrt(n);
    return (temp * temp != n);
}
```

```
public static boolean what (int n)
{
    return (f(n)%2 == 0);
}

public static int f (int n)
{
    int c = 1;
    for (int i=2; i<=n; i++)
    {
        if (n%i == 0)
            c++;
    }
    return c;
}
```

א. מה מבצעת השיטה what בהינתן לה מספר שלם חיובי n כלשהו. הסבירו בקצרה מה מבצעת השיטה ולא כיצד היא מבצעת זאת. כלומר, כתבו במפורש מה המשמעות של הערך המוחזר מהשיטה what. (4 נק').

ב. מהי סיבוכיות הזמן של השיטה what ? (2 נק')

ג. כתבו את השיטה what כך שתבצע את מה שביצעה בסעיף א בסיבוכיות זמן ריצה קטנה יותר. (17 נק')

שימו לב:

השיטה שתכתבו צריכה להיות יעילה ככל הניתן, גם מבחינת סיבוכיות הזמן וגם מבחינת סיבוכיות המקום. תשובה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבעיה תקבל מעט נקודות בלבד.

ד. מה סיבוכיות זמן הריצה של השיטה שכתבתם בסעיף ג? הסבירו תשובתכם. (2 נק')

אל תשכחו לתעד את מה שכתבתם!

```
private static int f (int[]a, int low, int high)
{
    int res = 0;
    for (int i=low; i<=high; i++)
        res += a[i];
    return res;
}

public static boolean what (int[]a, int num)
{
    for (int i=0; i<a.length; i++)
    {
        for (int j=i; j<a.length; j++)
        {
            if (f(a, i, j) == num)
                return true;
        }
    }
    return false;
}
```

- א. מה מבצעת השיטה what בחינתן לה מערך a מלא במספרים שלמים (חיוביים בלבד)? הסבירו בקצרה מה מבצעת השיטה ולא כיצד היא מבצעת זאת. כלומר, כתבו מה המשמעות של הערך המוחזר מהשיטה what. (4 נק')
- ב. מה סיבוכיות הזמן והמקום הכוללת של השיטה what ? (2 נק')
- ג. כתבו את השיטה what כך שתבצע את מה שביצעה בסעיף א בסיבוכיות זמן ריצה קטנה יותר. (17 נק')

שימו לב:

השיטה שתכתבו צריכה להיות יעילה ככל הניתן, גם מבחינת סיבוכיות הזמן וגם מבחינת סיבוכיות המקום. תשובה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבעיה תקבל מעט

נקודות בלבד.

- ד. מה סיבוכיות זמן הריצה והמקום של השיטה שכתבתם בסעיף ג? הסבירו תשובתכם. (2 נק')

אל תשכחו לתעד את מה שכתבתם!

האם יש תת סידרה של מספרים רצופים במערך שסכומם הוא num (הפרמטר).

```
public static boolean betterWhat(int[] a, int num)
{
    int l = 0, r = 1, sum = a[l];

    while (l <= r) {
        if (sum == num)
        {
            System.out.println("a[" + l + "]=" + a[l] + "...a[" + r + "]=" + a[r]);
            return true;
        }
        if (sum < num)
        {
            if (r == a.length)
                return false;
            sum += a[r];
            r++;
        }
        else
        {
            sum -= a[l];
            l++;
        }
    }

    return false;
}
```

```
private static int f (int[]a, int low, int high)
{
    int res = 0;
    for (int i=low; i<=high; i++)
        res += a[i];
    return res;
}
```

```
public static boolean what (int[]a, int num)
{
    for (int i=0; i<a.length; i++)
    {
        for (int j=i; j<a.length; j++)
        {
            if (f(a, i, j) == num)
                return true;
        }
    }
    return false;
}
```

- א. מה מבצעת השיטה what בחינתן לה מערך a מלא במספרים שלמים (חיוביים בלבד)? הסבירו בקצרה מה מבצעת השיטה ולא כיצד היא מבצעת זאת. כלומר, כתבו מה המשמעות של הערך המוחזר מהשיטה what. (4 נק')
- ב. מה סיבוכיות הזמן והמקום הכוללת של השיטה what? (2 נק')
- ג. כתבו את השיטה what כך שתבצע את מה שביצעה בסעיף א בסיבוכיות זמן ריצה קטנה יותר. (17 נק')

שימו לב:

השיטה שתכתבו צריכה להיות יעילה ככל הניתן, גם מבחינת סיבוכיות הזמן וגם מבחינת סיבוכיות המקום. תשובה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבעיה תקבל מעט

נקודות בלבד.

- ד. מה סיבוכיות זמן הריצה והמקום של השיטה שכתבתם בסעיף ג? הסבירו תשובתכם. (2 נק')

אל תשכחו לתעד את מה שכתבתם!

שאלה דומה בממ"ן 14 סמסטר 19ב !

```
private static int f (int[]a, int low, int high)
{
    int res = 0;
    for (int i=low; i<=high; i++)
        res += a[i];
    return res;
}

public static int what (int []a)
{
    int temp = 0;
    for (int i=0; i<a.length; i++)
    {
        for (int j=i; j<a.length; j++)
        {
            int c = f(a, i, j);
            if (c%3 == 0)
            {
                if (j-i+1 > temp)
                    temp = j-i+1;
            }
        }
    }
    return temp;
}
```

א. מה מבצעת השיטה what בהינתן לה מערך a מלא במספרים שלמים (חיוביים ושיליים)? הסבירו בקצרה מה מבצעת השיטה ולא כיצד היא מבצעת זאת. כלומר, כתבו מה המשמעות של המספר המוחזר מהשיטה what. (4 נק')

ב. מהי סיבוכיות הזמן של השיטה what ? (2 נק')

ג. כתבו את השיטה what כך שתבצע את מה שביצעה בסעיף א בסיבוכיות זמן ריצה קטנה יותר. (17 נק')

שימו לב:

השיטה שתכתבו צריכה להיות יעילה ככל הניתן, גם מבחינת סיבוכיות הזמן וגם מבחינת סיבוכיות המקום. תשובה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבעיה תקבל מעט

נקודות בלבד.

ד. מה סיבוכיות זמן הריצה של השיטה שכתבתם בסעיף ג? הסבירו תשובתכם. (2 נק')

אל תשכחו לתעד את מה שכתבתם!


```
private final static int SUB_ARRAY_SUM_MODULO = 3;
public static int what2(int[] arr) {

    int sum = 0; //Used to summarize all the modulus of all the array cells
    int tempModVal;
    int lastIndexZeroNum = -1; // Stores the highest index when sum % 3 = 0
    int firstIndexOneNum = -1; // Stores the first index when sum % 3 = 1
    int firstIndexTwoNum = -1; // Stores the first index when sum % 3 = 2
    int lastIndexOneNum = -1; // Stores the last index when sum % 3 = 1
    int lastIndexTwoNum = -1; // Stores the last index when sum % 3 = 2

    //Iterate over the array list
    for (int i = 0; i < arr.length; i++)
    {
        // Calculate the remainder in order to summarize
        // the modules of each cell
        tempModVal = (sum + arr[i]) % SUB_ARRAY_SUM_MODULO;

        // In case mod value is negative add 3
        if (tempModVal < 0)
            tempModVal += SUB_ARRAY_SUM_MODULO;

        sum = tempModVal;

        //Saves the last index with sum remainder of 0
        if (sum == 0)
            lastIndexZeroNum = i;

        //Saves the first index with sum remainder of 1
        if (sum == 1 && firstIndexOneNum == -1)
            firstIndexOneNum = i;

        //Saves the first index with sum remainder of 2
        if (sum == 2 && firstIndexTwoNum == -1)
            firstIndexTwoNum = i;

        //Saves the last index with sum remainder of 1
        if (sum == 1 && firstIndexOneNum != -1 &&
            i != firstIndexOneNum)
            lastIndexOneNum = i;

        //Saves the last index with sum remainder of 2
        if (sum == 2 && firstIndexTwoNum != -1
            && i != firstIndexTwoNum)
            lastIndexTwoNum = i;
    }
}
```

```
int res = 0; // Initialize the result to 0 as a default

// If this index was found it means that all the cells from
// 0 to this index are dividable by 3 with no remainder.
if (lastIndexZeroNum > -1)
// Result is the found index with the addition of 1 because
// the count starts from 0
    res = lastIndexZeroNum + 1;

// When the occurrence of sum % 3 = 1 appears twice, the result
// will be the subtraction of the last index with the first index
if (lastIndexOneNum > -1)
    if (lastIndexOneNum - firstIndexOneNum > res)
        // Update the result only if larger than the last result
        res = lastIndexOneNum - firstIndexOneNum;

// When the occurrence of sum % 3 = 2 appears twice, the result
// will be the subtraction of the last index with the first index
if (lastIndexTwoNum > -1)
    if (lastIndexTwoNum - firstIndexTwoNum > res)
        // Update the result only if larger than the last result
        res = lastIndexTwoNum - firstIndexTwoNum;

return res;
}
```

```
private static int f (int[]a, int low, int high)
{
    int res = 0;
    for (int i=low; i<=high; i++)
        res += a[i];
    return res;
}

public static int what (int []a)
{
    int temp = 0;
    for (int i=0; i<a.length; i++)
    {
        for (int j=i; j<a.length; j++)
        {
            int c = f(a, i, j);
            if (c%3 == 0)
            {
                if (j-i+1 > temp)
                    temp = j-i+1;
            }
        }
    }
    return temp;
}
```

- מה מבצעת השיטה what בהינתן לה מערך a מלא במספרים שלמים (חיוביים ושליליים)? הסבירו בקצרה מה מבצעת השיטה ולא כיצד היא מבצעת זאת. כלומר, כתבו מה המשמעות של המספר המוחזר מהשיטה what. (4 נק')
 - מהי סיבוכיות הזמן של השיטה what ? (2 נק')
 - כתבו את השיטה what כך שתבצע את מה שביצעה בסעיף א בסיבוכיות זמן ריצה קטנה יותר. (17 נק')
- שימו לב: השיטה שתכתבו צריכה להיות יעילה ככל הניתן, גם מבחינת סיבוכיות הזמן וגם מבחינת סיבוכיות המקום. תשובה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבעיה תקבל מעט נקודות בלבד.
- מה סיבוכיות זמן הריצה של השיטה שכתבתם בסעיף ג? הסבירו תשובתכם. (2 נק')

אל תשכחו לתעד את מה שכתבתם!

מהו אורכה של סידרת המספרים הרצופה הארוכה ביותר במערך שסכומה מתחלק ב-3.

כתבו שיטה שחתימה היא

```
public static int subStrC(String s, char c)
```

המקבלת מחזרת s ותו c ומחזירה כמה תת-מחזרות יש ב- s שמתחילות ומסתיימות בתו c ויש **בתוכן** בדיוק פעם אחת את התו c .

לדוגמא, אם המחזרות היא $s = "abcbcabcbacabcc"$ והתו $c = 'c'$ התוצאה היא 4. (כי התת-מחזרות הן $"cbcbac", "cabcbac", "cacabcb", "cabcc"$ שימו לב שאינכם צריכים להחזיר את התת-מחזרות, אלא רק את מספרן.

סעיף ב. (17%)

כתבו שיטה שחתימה היא

```
public static int subStrMaxC(String s, char c, int k)
```

המקבלת מחזרות s , תו c ומספר שלם חיובי k , ומחזירה כמה תת-מחזרות יש ב- s שמתחילות ומסתיימות בתו c ויש **בתוכן** מקסימום k פעמים את התו c . שימו לב שאינכם צריכים להחזיר את התת-מחזרות, אלא רק את מספרן.

דוגמאות:

- אם המחזרות היא $s = "abcbcb"$, התו $c = 'c'$ והמספר $k=0$ התוצאה היא 1. (התת-מחזרות היא $"cbc"$)
- אם המחזרות היא $s = "abcbcabcbacab"$, התו $c = 'c'$ והמספר $k=2$ התוצאה היא 6. (התת-מחזרות הן $"cbcbac", "cabcbac", "cbcbac", "cabcbac", "cbcbac", "cbcbac"$)
- אם המחזרות היא $s = "abcbcabcbacab"$, התו $c = 'c'$ והמספר $k=3$ התוצאה היא שוב 6. (שכן התת-מחזרות הן $"cbcbac", "cabcbac", "cbcbac", "cabcbac", "cbcbac", "cbcbac"$ ואין אף תת-מחזרות המתחילה בתו c ומסתיימת בתו c ויש בתוכה 3 תווי c)
- אם המחזרות היא $s = "abc"$, התו $c = 'c'$ והמספר $k=2$ התוצאה היא 0, שכן אין אף תת-מחזרות שמתחילה ומסתיימת ב- c .

השיטות שתכתבו צריכות להיות יעילה ככל הניתן.

שיטה שתעבוד בסיבוכיות גבוהה מזו הנדרשת (במקום או בזמן) לא תקבל את מירב הנקודות. אל תשכחו לתעד את מה שכתבתם!

שימו לב, בפתרון הבעיות מותר להשתמש אך ורק בשיטות שלהלן (המוגדרות במחלקה `String`), ובפקודות השוואה. (אין חובה להשתמש בהן!)

- `public char charAt(int i)` - המחזירה את התו במקום ה- i במחזרות (עליה היא מופעלת)
- `public int length()` - המחזירה את אורך המחזרות עליה היא מופעלת.

```
public static int subStrC(String s, char c)
```

```
{
    int i;
    int nc = 0; // the number of occurrences of the character c in string s

    // count nc, the number of occurrences of the character c in string s
    for(i=0; i<s.length(); i++)
    {
        if(s.charAt(i) == c)
            nc++;
    }

    // if the number of c's is less than or equal to 2 then return 0
    if(nc <= 2)
        return 0;
    // otherwise, the number of substrings in s which start and end in c and contain exactly one c
    // is equal to nc - 2.
    else // if(nc > 2)
        return nc - 2;
}
```

```
public static int subStrMaxC(String s, char c, int K)
```

```
{
    int i, k;
    int nc = 0; // the number of occurrences of the character c in string s
    int numSubstr = 0;

    // count nc, the number of occurrences of the character c in string s
    for(i=0; i<s.length(); i++)
    {
        if(s.charAt(i) == c)
            nc++;
    }

    // if the number of c's is less than 2 then return 0
    if(nc < 2)
        return 0;
    else // if(nc >= 2)
    {
        // there can be at most nc-2 occurrences of c (excluding first and last c)
        if(K > nc - 2)
            K = nc - 2;

        for(k=0; k<=K; k++)
        {
            // the number of substrings in s which start and end in c and contain exactly k c's
            // is equal to nc - 1 - k
            numSubstr += nc - 1 - k;
        }

        // Alternatively
        numSubstr = (K + 1) * (nc - 1) - K * (K + 1) / 2;
    }

    return numSubstr;
}
```

נתון מערך חד-ממדי a המכיל ערכי 0 ו-1 בלבד.

לדוגמא:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	1	1	1	1	1	1	0	1	1	1	1	1	0	1	1

כתבו שיטה סטטית המקבלת כפרמטר מערך a כני"ל, ומחליפה כל ערך במערך **שאינו אפס** במרחק ממנו לנקודת האפס הקרובה ביותר (מימין או משמאל). שימו לב שב"מרחק" הכוונה פשוט למספר הצעדים במערך שיש לבצע על מנת להגיע לאיבר אפס הקרוב ביותר.

לדוגמא, עבור המערך a שלעיל, המערך a לאחר ההחלפה (כך שיכיל את המרחקים) יהיה:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	1	2	3	3	2	1	0	1	2	3	2	1	0	1	2

ניתן להניח שהמערך מכיל לפחות 0 אחד. כלומר, הוא אינו מכיל רק 1-ים.

חתימת השיטה היא:

```
public static void zeroDistance (int [] a)
```

שימו לב:

השיטה שתכתבו צריכה להיות יעילה ככל הניתן, גם מבחינת סיבוכיות הזמן וגם מבחינת סיבוכיות המקום. תשובה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבעיה תקבל מעט נקודות בלבד.

כתבו מה סיבוכיות הזמן וסיבוכיות המקום של השיטה שכתבתם.

```
public static void zeroDistance (int [] a)
{
```

```
//read from left to right
```

```
int pos=0, i=0;
```

```
while(i<a.length)
```

```
{
```

```
if (a[pos]==0 && a[i]!=0
```

```
    a[i]=i-pos;
```

```
else if(a[pos]!=0 && a[i]==0)
```

```
{
```

```
    a[pos]=i-pos;
```

```
    pos++;
```

```
    i--;
```

```
}
```

```
else if(a[i]==0)
```

```
    pos=i;
```

```
    i++;
```

```
}
```

```
//read from right to left
```

```
i=a.length-1;
```

```
pos=a.length-1;
```

```
while(i>=0)
```

```
{
```

```
if (a[pos]==0 && a[i]!=0)
```

```
{
```

```
    int x=pos-i;
```

```
    if(x<a[i])
```

```
        a[i]=x;
```

```
}
```

```
else if(a[i]==0)
```

```
    pos=i;
```

```
    i--;
```

```
}
```

```
} // zeroDistance
```

נתון מערך חד-ממדי a המכיל ערכי 0 ו-1 בלבד.

לדוגמא:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	1	1	1	1	1	1	0	1	1	1	1	1	0	1	1

כתבו שיטה סטטית המקבלת כפרמטר מערך a כני"ל, ומחליפה כל ערך במערך **שאינו אפס** במרחק ממנו לנקודת האפס הקרובה ביותר (מימין או משמאל). שימו לב שב"מרחק" הכוונה פשוט למספר הצעדים במערך שיש לבצע על מנת להגיע לאיבר אפס הקרוב ביותר.

לדוגמא, עבור המערך a שלעיל, המערך a לאחר ההחלפה (כך שיכיל את המרחקים) יהיה :

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
0	1	2	3	3	2	1	0	1	2	3	2	1	0	1	2

ניתן להניח שהמערך מכיל לפחות 0 אחד. כלומר, הוא אינו מכיל רק 1-ים.

חתימת השיטה היא :

```
public static void zeroDistance (int [] a)
```

שימו לב:

השיטה שתכתבו צריכה להיות יעילה ככל הניתן, גם מבחינת סיבוכיות הזמן וגם מבחינת סיבוכיות המקום. תשובה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבעיה תקבל מעט נקודות בלבד.

כתבו מה סיבוכיות הזמן וסיבוכיות המקום של השיטה שכתבתם.

חיפוש
בינארי

נתון מערך a ממיון המכיל מספרים שלמים שונים זה מזה.

נגדיר נקודת שבת (fixed point) במערך כאינדקס i שעבורו $a[i] = i$.

כתבו שיטה סטטית המקבלת כפרמטר מערך a מלא במספרים שלמים. השיטה צריכה למצוא את נקודת השבת במערך, אם היא נמצאת ולהחזיר אותה. אם אין נקודת שבת במערך, יוחזר -1 . אם יש יותר מנקודת שבת אחת, תוחזר אחת מהן באופן שרירותי.

לדוגמא, במערך a להלן נקודת השבת היא 2 כי $a[2] = 2$

0	1	2	3	4	5	6
-3	0	2	4	11	12	15

חתימת השיטה היא :

```
public static int fixedPoint (int [] a)
```

שימו לב:

השיטה שתכתבו צריכה להיות יעילה ככל הניתן. תשובה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבעיה תקבל מעט נקודות בלבד.

כתבו מה סיבוכיות הזמן וסיבוכיות המקום של השיטה שכתבתם.

אל תשכחו לתעד את מה שכתבתם!

```

public static int fixedPointLinearComplexity (int [] a)
{
    for (int i = 0; i < a.length; i++)
        if (i == a[i])
            return i;
    return -1;
}

```

```

public static int fixedPointLogarithmicComplexity (int [] a)
{
    int start = 0;
    int end = a.length-1;
    int mid;

    while(start <= end)
    {
        mid = (start+end)/2;

        if(a[mid] == mid)
            return mid;
        else if(a[mid] > mid)
            end = mid - 1;
        else
            start = mid+1;
    }
    return -1;
}

```

נתון מערך a ממיון המכיל מספרים שלמים שונים זה מזה.

נגדיר **נקודת שבת** (fixed point) במערך כאינדקס i שעבורו $a[i] = i$.

כתבו שיטה סטטית המקבלת כפרמטר מערך a מלא במספרים שלמים. השיטה צריכה למצוא את נקודת השבת במערך, אם היא נמצאת ולהחזיר אותה. אם אין נקודת שבת במערך, יוחזר -1 . אם יש יותר מנקודת שבת אחת, תוחזר אחת מהן באופן שרירותי.

לדוגמא, במערך a להלן נקודת השבת היא 2 כי $a[2] = 2$

0	1	2	3	4	5	6
-3	0	2	4	11	12	15

חתימת השיטה היא :

```
public static int fixedPoint (int [] a)
```

שימו לב:

השיטה שתכתבו צריכה להיות יעילה ככל הניתן. תשובה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבעיה תקבל מעט נקודות בלבד.

כתבו מה סיבוכיות הזמן וסיבוכיות המקום של השיטה שכתבתם.

אל תשכחו לתעד את מה שכתבתם!

שאלה 2 - 25 נקודות

מערך נקרא "ממוין לסרוגין" אם שני התת-מערכים שלו המורכבים אחד ממספרים המופיעים באינדקסים הזוגיים (0, 2, ...) והשני ממספרים המופיעים באינדקסים האי-זוגיים (1, 3, ...) הם ממוינים כל אחד. לדוגמא, המערך {1, 2, 5, 3, 6, 10, 9} הוא ממוין לסרוגין, בעוד שהמערך {1, 2, 5, 3, 4, 10, 9} הוא לא (כי 4 מופיע אחרי 5 בתת-מערך הזוגיים).

כתבו שיטה `findX` המקבלת מערך ממוין לסרוגין של שלמים `a` ומספר שלם `x` ומחזירה `true` אם קיימים ב-`a` שני איברים סמוכים שסכומם הוא `x`, ו-`false` אחרת. לדוגמא, עבור המערך {1, 2, 5, 3, 6, 10, 9} ו-`x=9`, השיטה תחזיר `true` (שני האיברים הסמוכים 3 ו-6 סכומם 9) ועבור אותו מערך `a` כאשר `x=5` השיטה תחזיר `false`. (אין שני איברים סמוכים שסכומם 5). יש להתייחס גם למקרה בו במערך יש רק איבר אחד, ואז השיטה צריכה להחזיר `false`.

השיטה צריכה להיות יעילה ככל האפשר מבחינת זמן ומבחינת מקום. שיטה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבעיה תקבל ניקוד חלקי בלבד.

אל תשכחו לתעד את מה שכתבתם!

כתבו מה סיבוכיות הזמן וסיבוכיות המקום של השיטה שכתבתם.

שימו לב: אסור להשתמש במחלקות מוכנות כבר של Java. מותר להשתמש במחלקות שניתנו בהרצאה ונמצאות בחוברת השקפים, ובלבד שתהיה הפניה מדויקת אליהן.

```

public static boolean findX(int[] a, int x)
{
    int low = 0, high = a.length-1, m, sum;
    while (low <= high)
    {
        m = (low+high)/2;
        sum = a[m]+a[m+1];

        if (sum == x)
            return true;

        if (sum < x)
            low = m+1;
        else
            high = m-1;
    }

    return false;
}

```

שאלה 2 - 25 נקודות

מערך נקרא "ממוין לסרוגין" אם שני התת-מערכים שלו המורכבים אחד ממספרים המופיעים באינדקסים הזוגיים (0, 2, ...) והשני ממספרים המופיעים באינדקסים האי-זוגיים (1, 3, ...) הם ממוינים כל אחד. לדוגמא, המערך {1, 2, 5, 3, 6, 10, 9} הוא ממוין לסרוגין, בעוד שהמערך {1, 2, 5, 3, 4, 10, 9} הוא לא (כי 4 מופיע אחרי 5 בתת-מערך הזוגיים).

כתבו שיטה findX המקבלת מערך ממוין לסרוגין של שלמים a ומספר שלם x ומחזירה true אם קיימים ב-a שני איברים סמוכים שסכומם הוא x, ו-false אחרת.

לדוגמא, עבור המערך {1, 2, 5, 3, 6, 10, 9} ו-x=9, השיטה תחזיר true (שני האיברים הסמוכים 3 ו-6 סכומם 9) ועבור אותו מערך a כאשר x=5 השיטה תחזיר false. (אין שני איברים סמוכים שסכומם 5). יש להתייחס גם למקרה בו במערך יש רק איבר אחד, ואז השיטה צריכה להחזיר false.

השיטה צריכה להיות יעילה ככל האפשר מבחינת זמן ומבחינת מקום. שיטה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבעיה תקבל ניקוד חלקי בלבד.

אל תשכחו לתעד את מה שכתבתם!

כתבו מה סיבוכיות הזמן וסיבוכיות המקום של השיטה שכתבתם.

שימו לב: אסור להשתמש במחלקות מוכנות כבר של Java. מותר להשתמש במחלקות שניתנו בהרצאה ונמצאות בחוברת השקפים, ובלבד שתהיה הפניה מדויקת אליהן.

נגדיר:

הזזה של s צעדים של מערך a היא העתקה של כל איבר באינדקס i לאינדקס $i+s$. ההזזה היא מעגלית. כלומר, אם המערך הוא בגודל n , וההזזה היא בגודל s , אז האיבר באינדקס $s-n$ עובר לאינדקס 0 וכך הלאה. שימו לב שאף איבר לא "מפסל" רחוקה. הוא מוזז ונרד לחוקה אחר ולא "נדרס".

נתון מערך ממויין בסדר עולה שעבר הזזה s כלשהי.

גודלו של s אינו ידוע. הוא יכול להיות גם אפס, כלומר שהמערך לא עבר הזזה כלל.

עליכם לכתוב שיטה סטטית בוליאנית שמקבלת כפרמטר מערך a ומספר sum ומחזירה `true` אם ישנם שני מספרים במערך שסכומם הוא sum , ו-`false` אחרת.

לדוגמא,

המערך a הוא:

0	1	2	3	4	5	6
65	70	-5	3	48	49	52

אם הפרמטר $sum = 44$, השיטה תחזיר `true` שכן $-5 + 49 = 44$.

אם הפרמטר $sum = 45$, השיטה תחזיר `false` שכן אין שני מספרים במערך שסכומם 45.

חתימת השיטה היא:

```
public static boolean findSum (int[] a, int sum)
```

שימו לב:

השיטה שתכתבו צריכה להיות יעילה ככל הניתן, גם מבחינת סיבוכיות הזמן וגם מבחינת סיבוכיות המקום. תשובה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבעיה תקבל מעט נקודות בלבד.

כתבו מה סיבוכיות הזמן וסיבוכיות המקום של השיטה שכתבתם.

אתם יכולים להניח שהמערך מלא במספרים שלמים ממוינים בסדר עולה שעבר הזזה כלשהי (אולי בגודל 0). אין צורך לבדוק זאת.

אל תשכחו לתעד את מה שכתבתם!

```

private static int findSmallestNumberLocation(int[] a)
{
    if (a[a.length - 1] > a[0])
        return 0;
    if (a[a.length - 1] < a[a.length - 2])
        return a.length - 1;

    int low = 0, high = a.length-1, mid = -1;
    while (low <= high)
    {
        mid = (low+high)/2;

        if (a[mid] < a[mid - 1] && a[mid] < a[mid + 1])
            return mid;

        if (a[mid] < a[0])
            high = mid-1;
        else
            low = mid+1;
    }
    return -1;
}

```

```

public static boolean findSum(int[] array, int sum)
{
    int shift = findSmallestNumberLocation(array);
    int bottom = 0, top = array.length - 1;

    while (bottom < top)
    {
        int shiftedBottom = (bottom + shift) % array.length;
        int shiftedTop = (top + shift) % array.length;
        if (array[shiftedBottom] + array[shiftedTop] == sum)
            return true;

        if (array[shiftedBottom] + array[shiftedTop] > sum)
            top--;
        else // (array[shiftedBottom] + array[shiftedTop] < value)
            bottom++;
    }
    return false;
}

```

נבדיר:

הזוה של s צעדים של מערך a היא העתקה של כל איבר באינדקס i לאינדקס $i+s$. ההזזה היא מעגלית. כלומר, אם המערך הוא בגודל n , וההזזה היא בגודל s , אז האיבר באינדקס $s-n$ עובר לאינדקס 0 וכך הלאה. שימו לב שאף איבר לא "מפסח" רחוקה. הוא מושזז ונרר לחזקוה אחר ולא "נדרס".

נתון מערך ממויין בסדר עולה שעבר הזזה s כלשהי.

גודלו של s אינו ידוע. הוא יכול להיות גם אפס, כלומר שהמערך לא עבר הזזה כלל.

עליכם לכתוב שיטה סטטית בוליאנית שמקבלת כפרמטר מערך a ומספר sum ומחזירה $true$ אם ישנם שני מספרים במערך שסכומם הוא sum , ו- $false$ אחרת.

לדוגמא,

המערך a הוא:

0	1	2	3	4	5	6
65	70	-5	3	48	49	52

אם הפרמטר $sum = 44$, השיטה תחזיר $true$ שכן $-5 + 49 = 44$

אם הפרמטר $sum = 45$, השיטה תחזיר $false$ שכן אין שני מספרים במערך שסכומם 45.

חתימת השיטה היא:

```
public static boolean findSum (int[] a, int sum)
```

שימו לב:

השיטה שתכתבו צריכה להיות יעילה ככל הניתן, גם מבחינת סיבוכיות הזמן וגם מבחינת סיבוכיות המקום. תשובה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבעיה תקבל מעט נקודות בלבד.

כתבו מה סיבוכיות הזמן וסיבוכיות המקום של השיטה שכתבתם.

אתם יכולים להניח שהמערך מלא במספרים שלמים ממוינים בסדר עולה שעבר הזזה כלשהי (אולי בגודל 0). אין צורך לבדוק זאת.

אל תשכחו לתעד את מה שכתבתם!

כתבו שיטה סטטית minAbsNum המקבלת מערך ממוין בסדר עולה של מספרים שלמים (חיוביים ושליליים) שונים זה מזה. השיטה צריכה להחזיר את הסכום המינימלי של ערכם המוחלט של שני איברים במערך $|a_i| + |a_j|$. ניתן להניח ש- $n > 1$.

שימו לב, אסור לשנות את תוכן המערך, גם לא באופן זמני.

לדוגמא, עבור הקלט הבא:

0	1	2	3	4	5	6	7	8	9
-1	3	5	7	8	11	17	25	30	36

השיטה תחזיר את הערך 4 כי $|-1| + |3| = 4$, ואין עוד זוג מספרים במערך שסכומם בערך מוחלט קטן מ-4.

חתימת השיטה היא:

```
public static int minAbsSum (int []a)
```

שימו לב:

השיטה שתכתבו צריכה להיות יעילה ככל הניתן, גם מבחינת סיבוכיות הזמן וגם מבחינת סיבוכיות המקום. תשובה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבעיה תקבל מעט נקודות בלבד.

כתבו מה סיבוכיות הזמן וסיבוכיות המקום של השיטה שכתבתם.

אל תשכחו לתעד את מה שכתבתם!

```

private static int findSmallestAbsoluteNumberLocation(int[] a)
{
    // Up to two negative numbers in the array
    if (a[0] > 0)
        return 0;
    if (a[1] > 0)
        return 1;
    // Up to two positive numbers in the array
    if (a[a.length - 1] < 0)
        return a.length - 1;
    if (a[a.length - 2] < 0)
        return a.length - 2;

    int low = 0, high = a.length-1, mid = -1;
    while (low <= high)
    {
        mid = (low+high)/2;

        if (Math.abs(a[mid]) <= Math.abs(a[mid - 1]) &&
            Math.abs(a[mid]) <= Math.abs(a[mid + 1]))
            return mid;

        if (a[mid] > 0)
            high = mid-1;
        else
            low = mid+1;
    }
    return -1;
}

```

```

public static int minAbsSum(int[] a)
{
    int smallestPositiveNumberLocation =
        findSmallestAbsoluteNumberLocation(a);

    // No negative numbers or only one negative number in the array
    if (smallestPositiveNumberLocation == 0)
        return a[0] + a[1];
    if (smallestPositiveNumberLocation == 1)
        return Math.abs(a[1]) + Math.min(Math.abs(a[0]), a[2]);

    // No positive numbers or only one positive number in the array
    if (smallestPositiveNumberLocation == a.length - 1)
        return Math.abs(a[a.length - 1]) + Math.abs(a[a.length - 2]);
    if (smallestPositiveNumberLocation == a.length - 2)
        return Math.abs(a[a.length - 2]) +
            Math.min(Math.abs(a[a.length - 1]),
                Math.abs(a[a.length - 3]));

    // There are three options to check :
    // sum of two biggest negative numbers,
    // sum of two positive numbers,
    // sum of one negative and one positive number.
    int option2 = Math.abs(a[smallestPositiveNumberLocation-1]) +
        Math.abs(a[smallestPositiveNumberLocation]);
    int option3 = Math.abs(a[smallestPositiveNumberLocation]) +
        Math.abs(a[smallestPositiveNumberLocation+1]);
    int option1 = Math.abs(a[smallestPositiveNumberLocation-2]) +
        Math.abs(a[smallestPositiveNumberLocation-1]);
    return Math.min(option1, Math.min(option2, option3));
}

```

כתבו שיטה סטטית minAbsSum המקבלת מערך ממוין בסדר עולה של מספרים שלמים (חיוביים ושליילים) שונים זה מזה. השיטה צריכה לחזור את הסכום המינימלי של ערכים המוחלט של שני איברים במערך $|a_i| + |a_j|$.

ניתן להניח ש- $n > 1$.

שימו לב, אסור לשנות את תוכן המערך, גם לא באופן זמני.

לדוגמא, עבור הקלט הבא:

0	1	2	3	4	5	6	7	8	9
-1	3	5	7	8	11	17	25	30	36

השיטה תחזיר את הערך 4 כי $|-1| + |3| = 4$, ואין עוד זוג מספרים במערך שסכומם בערך מוחלט קטן מ-4.

חתימת השיטה היא:

```
public static int minAbsSum (int []a)
```

שימו לב:

השיטה שתכתבו צריכה להיות יעילה ככל הניתן, גם מבחינת סיבוכיות הזמן וגם מבחינת סיבוכיות המקום. תשובה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבעיה תקבל מעט נקודות בלבד.

כתבו מה סיבוכיות הזמן וסיבוכיות המקום של השיטה שכתבתם.

אל תשכחו לתעד את מה שכתבתם!

נתון מערך מלא במספרים שלמים, שבו כל מספר מופיע פעמיים ברצף פרט למספר אחד שמופיע רק פעם אחת. המערך אינו ממוין.

לדוגמא, המערכים הבאים מקיימים את התנאי:

	0	1	2	3	4	5	6	7	8
a =	6	6	18	18	-4	-4	12	9	9

	0	1	2	3	4	5	6	7	8	9	10	11	12
b =	8	8	-7	-7	3	3	0	0	10	10	5	5	4

	0
c =	5

כתבו שיטה סטטית שמקבלת כפרמטר מערך שמקיים את התנאי הנ"ל, ומחזירה את המספר שמופיע במערך רק פעם אחת.
לדוגמא, במערכים לעיל:

- המספר הבודד במערך a הוא 12 שנמצא באינדקס 6
- המספר הבודד במערך b הוא 4 שנמצא באינדקס 12
- המספר הבודד במערך c הוא 5 שנמצא באינדקס 0

אתם יכולים להניח שהמערך אינו ריק ושהוא מקיים את התנאי, אין צורך לבדוק זאת.

חתימת השיטה היא:

```
public static int findSingle (int [] a)
```

שימו לב:

השיטה שתכתבו צריכה להיות יעילה ככל הניתן, גם מבחינת סיבוכיות הזמן וגם מבחינת סיבוכיות המקום. תשובה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבעיה תקבל מעט נקודות בלבד.

מה סיבוכיות זמן הריצה של השיטה שכתבתם? הסבירו תשובתכם.

אל תשכחו לתעד את מה שכתבתם!

```

public static int findSingle(int[] a)
{
    if (a.length == 1)
        return a[0];

    if (a[0] != a[1])
        return a[0];

    if (a[a.length - 1] != a[a.length - 2])
        return a[a.length - 1];

    int low = 0, high = a.length - 1;
    while (true)
    {
        iterationsNum++;
        int mid = (low + high) / 2;
        if (mid % 2 == 0)
        {
            if (a[mid] == a[mid + 1])
                low = mid + 2;
            else if (a[mid] != a[mid - 1])
                return a[mid];
            else
                high = mid - 2;
        }
        else // (mid % 2 == 1)
        {
            if (a[mid] == a[mid - 1])
                low = mid + 1;
            else if (a[mid] != a[mid + 1])
                return a[mid];
            else
                high = mid - 1;
        }
    }
}

```

נתון מערך מלא במספרים שלמים, שבו כל מספר מופיע פעמיים ברצף פרט למספר אחד שמופיע רק פעם אחת. המערך אינו ממוין.

לדוגמא, המערכים הבאים מקיימים את התנאי:

	0	1	2	3	4	5	6	7	8
a =	6	6	18	18	-4	-4	12	9	9

	0	1	2	3	4	5	6	7	8	9	10	11	12
b =	8	8	-7	-7	3	3	0	0	10	10	5	5	4

	0
c =	5

כתבו שיטה סטטית שמקבלת כפרמטר מערך שמקיים את התנאי הנ"ל, ומחזירה את המספר שמופיע במערך רק פעם אחת.
לדוגמא, במערכים לעיל:

- המספר הבודד במערך a הוא 12 שנמצא באינדקס 6
- המספר הבודד במערך b הוא 4 שנמצא באינדקס 12
- המספר הבודד במערך c הוא 5 שנמצא באינדקס 0

אתם יכולים להניח שהמערך אינו ריק ושהוא מקיים את התנאי, אין צורך לבדוק זאת.

חתימת השיטה היא:

```
public static int findSingle (int [] a)
```

שימו לב:

השיטה שתכתבו צריכה להיות יעילה ככל הניתן, גם מבחינת סיבוכיות הזמן וגם מבחינת סיבוכיות המקום. תשובה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבעיה תקבל מעט נקודות בלבד.

מה סיבוכיות זמן הריצה של השיטה שכתבתם? הסבירו תשובתכם.

אל תשכחו לתעד את מה שכתבתם!

כתבו שיטה סטטית המקבלת כפרמטרים מערך **ממוין** a של מספרים שלמים, ומספר שלם נוסף x.

השיטה צריכה להחזיר את מספר המופעים של המספר x במערך a.

חתימת השיטה היא:

```
public static int count (int []a, int x)
```

לדוגמא, עבור המערך

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
-5	-5	1	1	1	1	1	1	1	1	2	2	2	2	2	3	3	3	67	67	99

והמספר x = -5 השיטה תחזיר את הערך 2.

עבור אותו מערך והמספר x = 2 השיטה תחזיר את הערך 5.

עבור אותו מערך והמספר x = 8 השיטה תחזיר את הערך 0.

שימו לב:

- השיטה שתכתבו צריכה להיות יעילה ככל הניתן. תשובה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבעיה (במקום או בזמן) תקבל מעט נקודות בלבד.
- אל תשכחו לתעד את השיטה שכתבתם.
- כתבו מה סיבוכיות הזמן וסיבוכיות המקום של השיטה שכתבתם.
- אפשר להניח שהמערך אינו null ואינו ריק.

```

public static int count(int [] a, int x)
{
    int low = 0;
    int high = a.length-1;
    int mid=0;
    int oldLow=0, oldHigh=0;
    boolean hit = false;

    // find the first occurrence of x

    while (low <= high && !hit)
    {
        oldHigh = high;
        oldLow = low;
        mid = (low+high)/2;
        if (a[mid] == x)
            hit = true;
        if (a[mid] > x)
            high = mid -1;
        else
            low = mid +1;
    }
    if (!hit)
        return 0;
}

```

```

// finds all the numbers that are identical to
// X and are to the left of the found index

low = oldLow;
high = mid;
int mid1=0;
hit = false;
if (a[0] == x)
    mid1 = 0;
else
{
    while (low <= high && !hit)
    {
        mid1 = (low+high)/2;
        if ((a[mid1] == x) && (a[mid1-1] != x))
            hit = true;
        if (a[mid1] > x)
            high = mid1 -1;
        else
            low = mid1 +1;
    } // while
} // else

```

```

// find all the numbers that are identical to
// X and are to the right of the found index

low = mid;
high = oldHigh;
int mid2=0;
hit = false;
if (a[a.length-1] == x)
    mid2 = a.length-1;
else
{
    while (low <= high && !hit)
    {
        mid2 = (low+high)/2;
        if ((a[mid2] == x) && (a[mid2+1] != x))
            hit = true;
        if (a[mid2] > x)
            high = mid2 -1;
        else
            low = mid2 +1;
    } // while
} // else

```

```

return mid2 - mid1 + 1;
}

```

כתבו שיטה סטטית המקבלת כפרמטרים מערך ממיון a של מספרים שלמים, ומספר שלם נוסף x.

השיטה צריכה להחזיר את מספר המופעים של המספר x במערך a.

חתימת השיטה היא:

```
public static int count (int []a, int x)
```

לדוגמא, עבור המערך

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
-5	-5	1	1	1	1	1	1	1	1	2	2	2	2	2	3	3	3	67	67	99

והמספר x = -5 השיטה תחזיר את הערך 2.

עבור אותו מערך והמספר x = 2 השיטה תחזיר את הערך 5.

עבור אותו מערך והמספר x = 8 השיטה תחזיר את הערך 0.

שימו לב:

- השיטה שתכתבו צריכה להיות יעילה ככל הניתן. תשובה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבעיה (במקום או בזמן) תקבל מעט נקודות בלבד.

- אל תשכחו לתעד את השיטה שכתבתם.

- כתבו מה סיבוכיות הזמן וסיבוכיות המקום של השיטה שכתבתם.

- אפשר להניח שהמערך אינו null ואינו ריק.

מעבר
לינארי

נתונים שני מערכים a ו- b המכילים מספרים שלמים כל אחד מהמערכים ממוין בסדר לא יורד (יכולים להיות מספרים זהים).

עליכם לכתוב שיטה סטטית שמקבלת שני מערכים ממוינים בסדר לא יורד, ומספר x שלם. השיטה צריכה למצוא זוג איברים במערכים, שהסכום שלהם הוא הקרוב ביותר ל- x (לא משנה אם הוא גדול מ- x או קטן מ- x). בזוג האיברים צריך להיות איבר אחד מכל מערך.

דוגמאות

נניח שהמערכים הם: $a = \{0, 4, 6, 11, 11\}$ ו- $b = \{10, 20, 30, 40\}$

- אם $x = 11$, הפלט צריך להיות 0 and 10
- אם $x = 13$, הפלט צריך להיות 4 and 10
- אם $x = 18$, הפלט יכול להיות 0 and 20 או 10 and 6 (לא משנה)
- אם $x = 70$, הפלט צריך להיות 40 and 11

חתימת השיטה היא:

```
public static void printClosest(int[] a, int[] b, int x)
```

שימו לב:

השיטה שתכתבו צריכה להיות יעילה ככל הניתן, גם מבחינת סיבוכיות הזמן וגם מבחינת סיבוכיות המקום. תשובה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבעיה תקבל מעט נקודות בלבד.

כתבו מה סיבוכיות הזמן וסיבוכיות המקום של השיטה שכתבתם.

אל תשכחו לתעד את מה שכתבתם!

```

public static int printClosestSquareComplexity(int[] a, int[] b, int x)
{
    int min = Integer.MAX_VALUE;
    int minA = 0, minB = 0;

    for (int aIndex = 0; aIndex < a.length; aIndex++)
        for (int bIndex = 0; bIndex < b.length; bIndex++)
            if (Math.abs(a[aIndex] + b[bIndex] - x) < min)
            {
                min = Math.abs(a[aIndex] + b[bIndex] - x);
                minA = aIndex;
                minB = bIndex;
            }

    System.out.println("printClosestSquareComplexity x is : " + x + " min is : " + min);
    System.out.println("In a : location - " + minA + " value - " + a[minA]);
    System.out.println("In b : location - " + minB + " value - " + b[minB]);
    return min;
}

```

```

public static int printClosestLinearComplexity(int[] a, int[] b, int x)
{
    int aIndex = 0, bIndex = b.length - 1;
    int min = Integer.MAX_VALUE;
    int minA = 0, minB = 0;

    while (aIndex < a.length && bIndex > -1)
    {
        if (Math.abs(a[aIndex] + b[bIndex] - x) < min)
        {
            min = Math.abs(a[aIndex] + b[bIndex] - x);
            minA = aIndex;
            minB = bIndex;
            if (min == 0)
                break;
        }
        if (a[aIndex] + b[bIndex] > x)
            bIndex--;
        else // (array[aIndex] + array[bIndex] < value)
            aIndex++;
    }

    System.out.println("printClosestLinearComplexity x is : " + x + " min is : " + min);
    System.out.println("In a : location - " + minA + " value - " + a[minA]);
    System.out.println("In b : location - " + minB + " value - " + b[minB]);
    return min;
}

```

נתונים שני מערכים a ו-b המכילים מספרים שלמים כל אחד מהמערכים ממוין בסדר לא יורד (יכולים להיות מספרים זהים).

עליכם לכתוב שיטה סטטית שמקבלת שני מערכים ממוינים בסדר לא יורד, ומספר x שלם. השיטה צריכה למצוא זוג איברים במערכים, שהסכום שלהם הוא הקרוב ביותר ל-x (לא משנה אם הוא גדול מ-x או קטן מ-x). בזוג האיברים צריך להיות איבר אחד מכל מערך.

דוגמאות

נניח שהמערכים הם: a = {0, 4, 6, 11, 11} ו-b = {10, 20, 30, 40}

- אם x = 11, הפלט צריך להיות 0 and 10
- אם x = 13, הפלט צריך להיות 4 and 10
- אם x = 18, הפלט יכול להיות 0 and 20 או 10 and 6 (לא משנה)
- אם x = 70, הפלט צריך להיות 40 and 11

חתימת השיטה היא:

```
public static void printClosest(int[] a, int[] b, int x)
```

שימו לב:

השיטה שתכתבו צריכה להיות יעילה ככל הניתן, גם מבחינת סיבוכיות הזמן וגם מבחינת סיבוכיות המקום. תשובה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבעיה תקבל מעט נקודות בלבד.

כתבו מה סיבוכיות הזמן וסיבוכיות המקום של השיטה שכתבתם.

אל תשכחו לתעד את מה שכתבתם!

כתבו שיטה סטטית בוליאנית **יעילה** המקבלת כפרמטר מערך חד-ממדי arr המכיל מספרים

שלמים. **המערך ממוין בסדר עולה ממש**. הערך 0 לא נמצא במערך.

השיטה צריכה להחזיר true אם קיימים במערך arr שני אינדקסים limit1 ו-limit2 שונים כך שמתקיימים התנאים הבאים:

- א. $limit1 < limit2$
- ב. סכום הערכים בתאים מ-0 עד limit1 הוא שלילי.
- ג. סכום הערכים בתאים מ-limit1+1 עד limit2 הוא אפס.
- ד. סכום הערכים בתאים מ-limit2+1 עד arr.length-1 הוא חיובי.

השיטה תחזיר false אם אין אינדקסים כאלו.

לדוגמא, במערך arr הבא

0	1	2	3	4	5	6
-8	-7	-5	-3	-2	5	8

יש שני אינדקסים $limit1 = 2, limit2 = 5$ המקיימים את התנאים:

- א. $2 < 5$
- ב. סכום הערכים בתאים 0 עד 2 הוא $(-8) + (-7) + (-5) = (-20)$ כלומר שלילי
- ג. סכום הערכים בתאים 3 עד 5 הוא $(-3) + (-2) + 5 = 0$ כלומר אפס
- ד. סכום הערכים בתאים 6 עד 6 הוא 8 כלומר חיובי

ולכן השיטה תחזיר את הערך true.

שימו לב שאין צורך להחזיר את האינדקסים עצמם אלא רק true אם יש כאלו אינדקסים או false אם אין.

חתימת השיטה:

```
public static boolean splitTo3 (int [] arr)
```

דוגמאות נוספות:

• במערך

0	1	2	3	4	5	6
-8	-7	-5	-3	-2	7	8

אין אינדקסים המקיימים את התנאים, ולכן השיטה תחזיר false.

• במערך

0	1	2	3	4	5	6
-8	-7	-5	-3	-2	-1	1

החלק עם הסכום החיובי הוא "ריק", אבל השיטה תחזיר true

• כך גם במערך

0	1	2	3	4	5	6
-8	-7	15	23	32	37	38

החלק עם הסכום השלילי הוא "ריק", אבל השיטה תחזיר true

שימו לב:

- השיטה שתכתבו צריכה להיות יעילה ככל הניתן, גם מבחינת סיבוכיות הזמן וגם מבחינת סיבוכיות המקום. תשובה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבנייה תקבל טעט נקודות בלבד.
- כתבו מה סיבוכיות הזמן ומה סיבוכיות המקום של כל שיטה שכתבתם.
- אל תשיכו לחזק את מה שכתבתם!

```

public static boolean splitTo3(int[] arr)
{
    if (arr.length < 2)
        return false;

    int limit1 = 0, limit2 = arr.length-1, sum = 0;

    for (int i = limit1; i < limit2; i++)
        sum += arr[i];

    while (limit1 < limit2)
    {
        if (sum == 0)
            return true;
        else if (sum < 0)
        {
            sum -= arr[limit1];
            limit1++;
        }
        else if (sum > 0)
        {
            sum -= arr[limit2];
            limit2--;
        }
    }

    return false;
}

```

שימו לב שאין צורך להחזיר את האינדקסים עצמם אלא רק true אם יש פתרון אינדקסים או false אם אין.

חתימת השיטה:

```
public static boolean splitTo3 (int [] arr)
```

דוגמאות נוספות:

• במערך

0	1	2	3	4	5	6
-8	-7	-5	-3	-2	7	8

אין אינדקסים המקיימים את התנאים, ולכן השיטה תחזיר false.

• במערך

0	1	2	3	4	5	6
-8	-7	-5	-3	-2	-1	1

החלק עם הסכום החיובי הוא "יריק", אבל השיטה תחזיר true

• כך גם במערך

0	1	2	3	4	5	6
-8	-7	15	23	32	37	38

החלק עם הסכום השלילי הוא "יריק", אבל השיטה תחזיר true

שימו לב:

- השיטה שתכתבו צריכה להיות יעילה ככל הניתן, גם מבחינת סיבוכיות הזמן וגם מבחינת סיבוכיות המקום. תשובה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבנייה תקבל טעם נקודות בלבד.
- כתבו מה סיבוכיות הזמן ומה סיבוכיות המקום של כל שיטה שכתבתם.
- אל תשיכו לחזק את מה שכתבתם!

כתבו שיטה סטטית בוליאנית **יעילה** המקבלת כפרמטר מערך חד-ממדי arr המכיל מספרים שלמים. המערך ממוין בסדר עולה ממש. הערך 0 לא נמצא במערך.

השיטה צריכה להחזיר true אם קיימים במערך arr שני אינדקסים limit1 ו-limit2 שונים כך שמתקיימים התנאים הבאים:

א. $limit1 < limit2$

ב. סכום הערכים בתאים מ-0 עד limit1 הוא שלילי.

ג. סכום הערכים בתאים מ-limit1+1 עד limit2 הוא אפס.

ד. סכום הערכים בתאים מ-limit2+1 עד arr.length-1 הוא חיובי.

השיטה תחזיר false אם אין אינדקסים כאלו.

לדוגמה, במערך arr הבא

0	1	2	3	4	5	6
-8	-7	-5	-3	-2	5	8

יש שני אינדקסים limit1 = 2, limit2 = 5 המקיימים את התנאים:

א. $2 < 5$

ב. סכום הערכים בתאים 0 עד 2 הוא $(-8) + (-7) + (-5) = (-20)$ כלומר שלילי

ג. סכום הערכים בתאים 3 עד 5 הוא $(-3) + (-2) + 5 = 0$ כלומר אפס

ד. סכום הערכים בתאים 6 עד 6 הוא 8 כלומר חיובי

ולכן השיטה תחזיר את הערך true.

שאלה 2 - 25 נקודות

נתון מערך a מלא במספרים שלמים .

עליכם לכתוב שיטה סטטית בשם replace המקבלת את המערך הזה כפרמטר, ומחליפה כל איבר במערך באיבר הגדול ביותר שנמצא מימינו במערך. האיבר האחרון יוחלף ב-0 כיוון שאין אף איבר מימין לו.

לדוגמא, אם המערך a היה:

0	1	2	3	4	5
6	7	4	3	5	2

לאחר הקריאה לשיטה replace המערך a ייראה כך:

0	1	2	3	4	5
7	5	5	5	2	0

חתימת השיטה היא:

```
public static void replace (int [] a)
```

שימו לב:

השיטה שתכתבו צריכה להיות יעילה ככל הניתן, גם מבחינת סיבוכיות הזמן וגם מבחינת סיבוכיות המקום. תשובה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבעיה תקבל מעט נקודות בלבד.

מה סיבוכיות זמן הריצה והמקום של השיטה שכתבתם? הסבירו תשובתכם. (2 נק')

אל תשכחו לתעד ברור את מה שכתבתם!

שאלה 2 - 25 נקודות

נתון מערך a מלא במספרים שלמים.

עליכם לכתוב שיטה סטטית בשם replace המקבלת את המערך הזה כפרמטר, ומחליפה כל איבר במערך באיבר הגדול ביותר שנמצא מימינו במערך. האיבר האחרון יוחלף ב-0 כיוון שאין אף איבר מימין לו.

לדוגמא, אם המערך a היה:

0	1	2	3	4	5
6	7	4	3	5	2

לאחר הקריאה לשיטה replace המערך a ייראה כך:

0	1	2	3	4	5
7	5	5	5	2	0

חתימת השיטה היא:

```
public static void replace (int [] a)
```

שימו לב:

השיטה שתכתבו צריכה להיות יעילה ככל הניתן, גם מבחינת סיבוכיות הזמן וגם מבחינת סיבוכיות המקום. תשובה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבעיה תקבל מעט נקודות בלבד.

מה סיבוכיות זמן הריצה והמקום של השיטה שכתבתם? הסבירו תשובתכם. (2 נק')

אל תשכחו לתעד ברור את מה שכתבתם!

```
public static void replace(int[] a)
{
    int len = a.length-1;
    int maxOfNumbersToTheRight = 0;

    if (len == -1)
        a[0] = 0;

    maxOfNumbersToTheRight = Math.max(a[len-1], a[len]);
    a[len-1] = a[len];
    a[len] = 0;

    for (int i = len-2; i >= 0; i--)
    {
        int temp = a[i];
        a[i] = maxOfNumbersToTheRight;
        maxOfNumbersToTheRight = Math.max(maxOfNumbersToTheRight, temp);
    }

    for (int i = 0; i < a.length; i++)
        System.out.println("a " + i + " " + a[i]);
}
```

נתון מערך a המלא במספרים שלמים ממוין בסדר עולה ממש. (אין צורך לבדוק זאת).
כתבו שיטה סטטית המקבלת את המערך כפרמטר וכן מספר שלם חיובי k. השיטה צריכה
להדפיס את כל הזוגות במערך שהחפרש ביניהם הוא בדיוק k. (אין צורך לבדוק ש-k חיובי).
שימו לב, מדובר בהפרשים בין הערכים שבתאים ולא בין האינדקסים.

חתימת השיטה היא:

```
public static void printPairs (int []a, int k)
```

לדוגמא,

אם המערך a הוא

0	1	2	3	4	5	6	7	8	9	10	11
-7	-3	0	1	3	5	12	14	17	19	25	30
						2	5	2	6	6	

אזי אם $k = 2$ יודפס:

Pair Found: (1,3)

Pair Found: (3,5)

Pair Found: (12,14)

Pair Found: (17,19)

אם $k = 6$ יודפס:

Pair Found: (-3,3)

Pair Found: (19,25)

ואם $k = 23$ לא יודפס כלום כי אין אף זוג איברים במערך שהחפרש בין ערכיו הוא 23.

שימו לב:

השיטה שתכתבו צריכה להיות יעילה ככל הניתן, גם מבחינת סיבוכיות הזמן וגם
מבחינת סיבוכיות המקום. תשובה שאינה יעילה מספיק כלומר, שתהיה
בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבעיה תקבל מעט נקודות בלבד.

מה סיבוכיות זמן הריצה והמקום של השיטה שכתבתם! הסבירו תשובתכם. (2 נק')

אל תשכחו לתעד ברור את מה שכתבתם!

נתון מערך a המלא במספרים שלמים ממוין בסדר עולה ממש. (אין צורך לבדוק זאת).

כתבו שיטה סטטית המקבלת את המערך כפרמטר וכן מספר שלם חיובי k. השיטה צריכה להדפיס את כל הזוגות במערך שהחפרש ביניהם הוא בדיוק k. (אין צורך לבדוק ש-k חיובי). שימו לב, מדובר בהפרשים בין הערכים שבתאים ולא בין האינדקסים.

חתימת השיטה היא:

```
public static void printPairs (int []a, int k)
```

לדוגמא,

אם המערך a הוא

0	1	2	3	4	5	6	7	8	9	10	11
-7	-3	0	1	3	5	12	14	17	19	25	30

אזי אם $k = 2$ יודפס:

Pair Found: (1,3)

Pair Found: (3,5)

Pair Found: (12,14)

Pair Found: (17,19)

אם $k = 6$ יודפס:

Pair Found: (-3,3)

Pair Found: (19,25)

ואם $k = 23$ לא יודפס כלום כי אין אף זוג איברים במערך שהחפרש בין ערכיו הוא 23.

שימו לב:

השיטה שתכתבו צריכה להיות יעילה ככל הניתן, גם מבחינת סיבוכיות הזמן וגם מבחינת סיבוכיות המקום. תשובה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבעיה תקבל מעט נקודות בלבד.

מה סיבוכיות זמן הריצה והמקום של השיטה שכתבתם! הסבירו תשובתכם. (2 נק')

אל תשכחו לתעד ברור את מה שכתבתם!

```
public static void printPairs(int[] a, int k) {
    int l = 0;
    int r = 1;

    while (r < a.length)
    {
        if (a[r]-a[l] > k)
            l++;
        else if (a[r]-a[l] < k)
            r++;
        else
        {
            System.out.println("Pair Found: (" + a[l] + ", " + a[r] + ")");
            l++;
            r++;
        }
        if (r == l)
            r++;
    }
}
```

שאלה 2 - 25 נקודות

נתון מערך a המלא במספרים שלמים אי-שליליים (אין צורך לבדוק זאת). כתבו שיטה סטטית המקבלת את המערך כפרמטר וכן מספר שלם חיובי k . השיטה צריכה למצוא את התת-מערך (sub-array) הרצוף הקטן ביותר שהסכום של איבריו גדול מהמספר k , ולהחזיר את האורך של התת-מערך הזה. אם אין תת-מערך כזה (כלומר, הסכום של כל איברי המערך עדיין קטן מהמספר k), השיטה תחזיר את הערך $a.length+1$ (אורך המערך a ועוד 1).

חתימת השיטה היא:

```
public static void smallestSub (int []a, int k)
```

דוגמאות:

אם המערך a הוא

0	1	2	3	4	5
1	4	13	6	0	19

- אם $k = 22$ השיטה תחזיר את הערך 3 כי התת-מערך הקטן ביותר הוא $\{4, 13, 6\}$ או התת-מערך $\{6, 0, 19\}$.
- אם $k = 42$ השיטה תחזיר את הערך 6 כי התת-מערך הקטן ביותר הוא המערך כולו.
- אם $k = 43$ השיטה תחזיר את הערך 7 כי אין תת-מערך שסכום איבריו גדול מ-43.
- אם $k = 26$ השיטה תחזיר את הערך 4 כי התת-מערך הקטן ביותר הוא $\{13, 6, 0, 19\}$.
- אם $k = 2$ השיטה תחזיר את הערך 1 כי הסכום של התת-מערך $\{4\}$ גדול מ-2 (וכך גם לגבי התת-מערכים $\{13\}$ או $\{6\}$ או $\{19\}$).

עליכם להחזיר את אורך התת-מערך הקטן ביותר ולא את איברי התת-מערך הזה.

שימו לב:

השיטה שתכתבו צריכה להיות יעילה ככל הניתן, גם מבחינת סיבוכיות הזמן וגם מבחינת סיבוכיות המקום. תשובה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבעיה תקבל מעט נקודות בלבד.

מה סיבוכיות זמן הריצה והמקום של השיטה שכתבתם! הסבירו תשובתכם. (2 נק')

אל תשכחו לתעד ברור את מה שכתבתם!

שאלה 2 - 25 נקודות

נתון מערך a המלא במספרים שלמים אי-שליליים (אין צורך לבדוק זאת).

כתבו שיטה סטטית המקבלת את המערך כפרמטר וכן מספר שלם חיובי k . השיטה צריכה למצוא את התת-מערך (sub-array) הרצוף הקטן ביותר שהסכום של איבריו גדול מהמספר k , ולהחזיר את האורך של התת-מערך הזה. אם אין תת-מערך כזה (כלומר, הסכום של כל איברי המערך עדיין קטן מהמספר k), השיטה תחזיר את הערך $a.length+1$ (אורך המערך a ועוד 1).

חתימת השיטה היא:

```
public static void smallestSub (int []a, int k)
```

דוגמאות:

אם המערך a הוא

0	1	2	3	4	5
1	4	13	6	0	19

- אם $k = 22$ השיטה תחזיר את הערך 3 כי התת-מערך הקטן ביותר הוא $\{4, 13, 6\}$ או התת-מערך $\{6, 0, 19\}$.
- אם $k = 42$ השיטה תחזיר את הערך 6 כי התת-מערך הקטן ביותר הוא המערך כולו.
- אם $k = 43$ השיטה תחזיר את הערך 7 כי אין תת-מערך שסכום איבריו גדול מ-43.
- אם $k = 26$ השיטה תחזיר את הערך 4 כי התת-מערך הקטן ביותר הוא $\{13, 6, 0, 19\}$.
- אם $k = 2$ השיטה תחזיר את הערך 1 כי הסכום של התת-מערך $\{4\}$ גדול מ-2 (וכך גם לגבי התת-מערכים $\{13\}$ או $\{6\}$ או $\{19\}$).

עליכם להחזיר את אורך התת-מערך הקטן ביותר ולא את איברי התת-מערך הזה.

שימו לב:

השיטה שתכתבו צריכה להיות יעילה ככל הניתן, גם מבחינת סיבוכיות הזמן וגם מבחינת סיבוכיות המקום. תשובה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבעיה תקבל מעט נקודות בלבד.

מה סיבוכיות זמן הריצה והמקום של השיטה שכתבתם? הסבירו תשובתכם. (2 נק')

אל תשכחו לתעד ברור את מה שכתבתם!

```
public static void smallestSub(int[] a, int k)
{
    int l = 0, r = 1, sum = a[0], count = 1, min = a.length;

    while (l <= r)
    {
        if (sum <= k)
        {
            if (r == a.length)
                break;
            sum += a[r];
            r++;
            count++;
        }
        else
        {
            min = Math.min(min, count);
            sum -= a[l];
            l++;
            count--;
        }
    }

    System.out.println(l == 0 && r == a.length ? min+1 : min);
}
```


כתבו שיטה יעילה המקבלת כפרמטר מערך חד-ממדי `arr` המלא במספרים שלמים חיוביים הממוינים בסדר לא יורד. השיטה מחזירה מרו המספר החיובי הקטן ביותר שלא יכול להיות סכום של קבוצת מספרים מהמערך.

חתימת השיטה היא:

```
public static int findSmallest (int [] arr)
```

דוגמאות:

• עבור המערך `arr` הבא:

0	1	2	3	4	5
1	2	6	10	11	15

מרו המספר הקטן ביותר שכא ירוכב להיות סכום של חלק מאיברי המערך!

1 יכול להיות כי הוא הסכום של התא `arr[0]`. 2 יכול להיות כי הוא הסכום של התא `arr[1]`. 3 יכול להיות כי הוא הסכום של התאים `arr[0]` ו-`arr[1]`. 4 לא יכול להיות סכום של חלק מאיברי המערך (לאו דווקא רצופים). לכן השיטה תחזיר 4.

- עבור המערך `arr` הבא: $arr[] = \{1, 1, 1, 1\}$ השיטה תחזיר 5
- עבור המערך `arr` הבא: $arr[] = \{1, 1, 3, 4\}$ השיטה תחזיר 10
- עבור המערך `arr` הבא: $arr[] = \{1, 3, 5, 10, 20, 40\}$ השיטה תחזיר 2
- עבור המערך `arr` הבא: $arr[] = \{1, 2, 4, 10, 11, 15\}$ השיטה תחזיר 8

אפשר להניח שהמערך מלא במספרים שלמים חיוביים והוא ממוין בסדר לא יורד. אין צורך לבדוק זאת.

שימו לב:

- השיטה שתכתבו צריכה להיות יעילה ככל הניתן, גם מבחינת סיבוכיות הזמן וגם מבחינת סיבוכיות המקום. תשובה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבעיה תקבל מעט נקודות בלבד.
- כתבו מח סיבוכיות חזמן וסיבוכיות חמקום של השיטה שכתבתם.
- אל תשכחו לתעד את מה שכתבתם!

```

public static int findSmallest(int[] arr)
{
    if (arr.length == 0 || arr[0] > 1)
        return 1;

    int i, sum;
    for (i = 1, sum = arr[0]; i < arr.length; i++)
    {
        if (arr[i] - sum > 1)
            return sum + 1;
        sum += arr[i];
    }
    return sum + 1;
}

```

כתבו שיטה יעילה המקבלת כפרמטר מערך חד-ממדי arr המלא במספרים שלמים חיוביים הממוינים בסדר לא יורד. השיטה מחזירה מרו המספר החיובי הקטן ביותר שלא יכול להיות סכום של קבוצת מספרים מהמערך.

חתימת השיטה היא:

```
public static int findSmallest (int [] arr)
```

דוגמאות:

• עבור המערך arr הבא:

0	1	2	3	4	5
1	2	6	10	11	15

מרו המספר הקטן ביותר שכא ירוכב להיות סכום של חלק מאיברי המערך!

1 יכול להיות כי הוא הסכום של התא arr[0]. 2 יכול להיות כי הוא הסכום של התא arr[1]. 3 יכול להיות כי הוא הסכום של התאים arr[0] ו-arr[1]. 4 לא יכול להיות סכום של חלק מאיברי המערך (לאו דווקא רצופים). לכן השיטה תחזיר 4.

• עבור המערך arr הבא: arr[] = {1, 1, 1, 1} השיטה תחזיר 5

• עבור המערך arr הבא: arr[] = {1, 1, 3, 4} השיטה תחזיר 10

• עבור המערך arr הבא: arr[] = {1, 3, 5, 10, 20, 40} השיטה תחזיר 2

• עבור המערך arr הבא: arr[] = {1, 2, 4, 10, 11, 15} השיטה תחזיר 8

אפשר להניח שהמערך מלא במספרים שלמים חיוביים והוא ממוין בסדר לא יורד. אין צורך לבדוק זאת.

שימו לב:

• השיטה שתכתבו צריכה להיות יעילה ככל הניתן, גם מבחינת סיבוכיות הזמן וגם מבחינת סיבוכיות המקום. תשובה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבעיה תקבל מעט נקודות בלבד.

• כתבו מחר סיבוכיות הזמן וסיבוכיות המקום של השיטה שכתבתם.

• אל תשכחו לתעד את מה שכתבתם!

כתבו שיטה סטטית המקבלת כפרמטר מערך arr המלא במספרים שלמים, (חיוביים ושליליים).

השיטה צריכה למצוא שלשה של מספרים במערך (לאו דווקא סמוכים זה לזה) שהמכפלה שלהם היא המקסימלית. השיטה תדפיס את שלושת המספרים האלו ותחזיר את המכפלה המקסימלית.

לדוגמא:

- אם המערך arr מכיל את המספרים הבאים:

0	1	2	3	4
-4	1	-8	9	6

אזי תודפס השלשה: 9 -8 -4

ותוחזר המכפלה $9 * -8 * -4 = 288$

חתימת השיטה היא:

```
public static int findTriplet (int [] arr)
```

שימו לב:

השיטה שתכתבו צריכה להיות יעילה ככל הניתן, גם מבחינת סיבוכיות הזמן וגם מבחינת סיבוכיות המקום. תשובה שאינה יעילה מספיק כלומר, שתהיה בסיבוכיות גדולה יותר מזו הנדרשת לפתרון הבעיה תקבל מעט נקודות בלבד.

מה סיבוכיות זמן הריצה של השיטה שכתבתם? הסבירו תשובתכם.

אל תשכחו לתעד את מה שכתבתם!

```
public static int findTriplet (int [] arr)
```

```
{
```

```
int minNegative1 = 0, minNegative2 = 0,  
maxNegative1 = Integer.MIN_VALUE,  
maxNegative2 = Integer.MIN_VALUE,  
maxNegative3 = Integer.MIN_VALUE,  
maxPositive1 = -1, maxPositive2 = -1, maxPositive3 = -1;
```

```
int negativeCounter = 0, positiveCounter = 0;
```

```
for (int i = 0; i < arr.length; i++)
```

```
{
```

```
if (arr[i] > 0)
```

```
{
```

```
positiveCounter++;
```

```
if (arr[i] > maxPositive3)
```

```
{
```

```
maxPositive1 = maxPositive2;
```

```
maxPositive2 = maxPositive3;
```

```
maxPositive3 = arr[i];
```

```
}
```

```
else if (arr[i] > maxPositive2)
```

```
{
```

```
maxPositive1 = maxPositive2;
```

```
maxPositive2 = arr[i];
```

```
}
```

```
else if (arr[i] > maxPositive1)
```

```
maxPositive1 = arr[i];
```

```
}
```

```
else // arr[i] < 0 - no zeroes in the array
```

```
else // arr[i] < 0 - no zeroes in the array
```

```
{
```

```
negativeCounter++;
```

```
if (arr[i] < minNegative1)
```

```
{
```

```
minNegative2 = minNegative1;
```

```
minNegative1 = arr[i];
```

```
}
```

```
else if (arr[i] < minNegative2)
```

```
minNegative2 = arr[i];
```

```
if (arr[i] > maxNegative3)
```

```
{
```

```
maxNegative1 = maxNegative2;
```

```
maxNegative2 = maxNegative3;
```

```
maxNegative3 = arr[i];
```

```
}
```

```
else if (arr[i] > maxNegative2)
```

```
{
```

```
maxNegative1 = maxNegative2;
```

```
maxNegative2 = arr[i];
```

```
}
```

```
else if (arr[i] > maxNegative1)
```

```
maxNegative1 = arr[i];
```

```
}
```

```
}
```

תבנו שיטה סטטית המקבלת כפרמטר מערך arr המלא במספרים שלמים, (חיוביים ושליליים). השיטה צריכה למצוא שלשה של מספרים במערך (לאו דווקא סמוכים זה לזה) שהמכפלה שלהם היא המקסימלית. השיטה תדפיס את שלושת המספרים האלו ותחזיר את המכפלה המקסימלית.

לדוגמא:

• אם המערך arr מכיל את המספרים הבאים:

0	1	2	3	4
-4	1	-8	9	6

אזי תודפס השלשה: -4 -8 9

ותוחזר המכפלה $-4 * -8 * 9 = 288$

חתימת השיטה היא:

```
public static int findTriplet (int [] arr)
```

```
if (positiveCounter == 0)
```

```
return maxNegative1 * maxNegative2 * maxNegative3;
```

```
else if (positiveCounter <= 2)
```

```
return minNegative1 * minNegative2 * maxPositive3;
```

```
else if (negativeCounter <= 1)
```

```
return maxPositive1 * maxPositive2 * maxPositive3;
```

```
// positiveCounter >= 3 && negativeCounter >= 2
```

```
int twoNegativesOnePositive =
```

```
minNegative1 * minNegative2 * maxPositive3;
```

```
int threePositives =
```

```
maxPositive1 * maxPositive2 * maxPositive3;
```

```
if (twoNegativesOnePositive > threePositives)
```

```
return minNegative1 * minNegative2 * maxPositive3;
```

```
else
```

```
return maxPositive1 * maxPositive2 * maxPositive3;
```

```
}
```

נאמר ששורה של n מספרים שלמים מייצגת גבהים של נקודות ברכס, שיש בהם פסגות ועמקים.

כתבו שיטה שחתימתה:

```
public static int maximalDrop (int [] a)
```

המקבלת מערך ובו המספרים המייצגים את הגבהים האלו. השיטה צריכה להחזיר את הנפילה המקסימלית בין שני מספרים (לא בהכרח בתאים עוקבים), כך שהמספר הגבוה נמצא לפני המספר הנמוך במערך.

דוגמאות:

- עבור המערך: {5, 21, 3, 22, 12, 7, 27, 6, 4} התשובה שתוחזר תהיה 23 (שזה 27-4)
- עבור המערך: {5, 21, 3, 22, 12, 7, 26, 14} התשובה שתוחזר תהיה 18 (שזה 21-3)
- עבור המערך: {5, 15, 3, 22, 12, 7, 27, 14} התשובה שתוחזר תהיה 15 (שזה 22-7)

שימו לב,

השיטה צריכה להיות יעילה ככל הניתן. עליך לציין את סיבוכיות הזמן והמקום

של השיטה שלך בתוך ה-API.

שיטה שתעבוד בסיבוכיות גבוהה מזו הנדרשת (במקום או בזמן) לא תקבל את מירב הנקודות.

נאמר ששורה של n מספרים שלמים מייצגת גבהים של נקודות ברכס, שיש בהם פסגות ועמקים.

כתבו שיטה שחתימתה:

```
public static int maximalDrop (int [] a)
```

המקבלת מערך ובו המספרים המייצגים את הגבהים האלו. השיטה צריכה להחזיר את הנפילה המקסימלית בין שני מספרים (לא בהכרח בתאים עוקבים), כך שהמספר הגבוה נמצא לפני המספר הנמוך במערך.

דוגמאות:

- עבור המערך: {5, 21, 3, 22, 12, 7, 27, 6, 4} התשובה שתוחזר תהיה 23 (שזה 27-4)
- עבור המערך: {5, 21, 3, 22, 12, 7, 26, 14} התשובה שתוחזר תהיה 18 (שזה 21-3)
- עבור המערך: {5, 15, 3, 22, 12, 7, 27, 14} התשובה שתוחזר תהיה 15 (שזה 22-7)

שימו לב,

השיטה צריכה להיות יעילה ככל הניתן. עליך לציין את סיבוכיות הזמן והמקום

של השיטה שלך בתוך ה-API.

שיטה שתעבוד בסיבוכיות גבוהה מזו הנדרשת (במקום או בזמן) לא תקבל את מירב הנקודות.

```
public static int maximalDrop (int [] a)
{
    if (a.length < 2)
        return 0;
    int i = 1; // search index
    int max = 0; // max index
    int min = 0; //min index
    int maxFall = 0; //maximum Fall found
    while (i < a.length)
    { // keep searching until array ends
        if (a[i] > a[max]){ // if new max value found
            max = i;
            min = i; // replace both indexes
        }
        else if (a[i] < a[min]) // else if new min value found
            min = i; // replace it
        if (a[max] - a[min] > maxFall) // if new maximum fall found
            maxFall = a[max] - a[min]; // replace it
        i++;
    }
    return maxFall;
}
```